DevOps & Low-Code Development - Afterhealth Report

Alejandro Regidor Miguel Romero Sara Gutiérrez
Nigina Abdukhalimova Ibrahim AYDAR

2025-12-15

Contents

1 Cover Page **2**

2 Assignment 1: Problem Discovery & Initial Design **2**
2.1 Introduction . . . 2
2.2 Purpose of the Report . . . 2
2.3 Structure . . . 3
2.4 Problem Statement & Context . . . 3
2.5 Theoretical Alignment . . . 4
2.6 Business Process Analysis . . . 7
2.7 Database Connection . . . 9
2.8 AI Collaboration Log . . . 11
2.9 References . . . 11

3 Assignment 2 – From BPMN to Power Apps Prototype **12**
3.1 Introduction . . . 12
3.2 Mapping BPMN → Features & Data . . . 12
3.3 Development Plan . . . 15
3.4 Project Backlog – Healthcare Follow-up System . . . 18
3.5 App Prototype . . . 22
3.6 Reflection & Lessons Learned . . . 31
3.7 References . . . 33

4 Assignment 3 – DevOps Planning, Pipeline, and Automation **33**
4.1 DevOps Planning (Azure Boards) . . . 33
4.2 Planning and Collaboration with Azure Boards: . . . 42
4.3 Power Apps Pipeline Setup . . . 42
4.4 Automation with Power Automate . . . 47
4.5 Reflection . . . 48

5 Assignment 4 — Copilot Integration, Monitoring, Testing, and Production Deployment **49**
5.1 Copilot Feature 49
5.2 Testing and Monitoring 53
5.3 Final Version Deployment to Production 63
5.4 Reflection and Lessons Learned 65

6 Final Reflection **66**
6.1 Executive Summary 66
6.2 Future Work 66
6.3 Detailed Teamwork Reflection 67
6.4 Sign-off 73

1 Cover Page

- **Group Number**: Group 1
- **Student Names**: Alejandro Regidor Orellana, Miguel Romero Catellanos, Sara Gutiérrez Caja, Nigina Abdukhalimova and Ibrahim Aydar
- **Course**: DevOps and Low-Code Development in Cloud
- **Date**: 23/09/2025

2 Assignment 1: Problem Discovery & Initial Design

2.1 Introduction

2.2 Purpose of the Report

Healthcare systems face an increasing challenge in ensuring that patient care does not end at the moment of hospital discharge. Recovery and adherence to treatment are critical stages that often determine long-term health outcomes, yet many patients struggle with medication schedules, follow-up appointments, and lifestyle adjustments once they return home.

This project addresses these gaps by exploring the problem of post-discharge continuity of care and proposing an information system that supports patients through structured reminders, digital summaries, and active monitoring by healthcare professionals.

The purpose of this report is to present the first stage of the assignment, focused on **problem discovery and initial design**. The report has the goal of:

- Identifying a clear problem related to patient post-discharge care.
- Analyzing the stakeholders involved and their expectations.
- Explaining why the problem is relevant as an **Information Systems** case.

- Connecting the problem with theoretical frameworks such as **BIS categories** and the **DIKW model**.
- Comparing the current (AS-IS) process with the improved (TO-BE) process.
- Showing how **AI tools** supported the work during problem framing and process analysis.

2.3 Structure

The report is structured in the following way:

1. **Problem Statement & Context**: Explains the main problem, identifies stakeholders, and describes why it is an Information Systems problem.
2. **Theoretical Alignment**: Shows how the system fits in BIS categories, applies the DIKW model, and identifies the type of organizational change.
3. **Business Process Analysis**: Describes the AS-IS process, the proposed TO-BE process, and includes diagrams (Ishikawa and BPMN).
4. **AI Collaboration Log**: Records the use of AI prompts, the outputs obtained, and reflections on how they contributed to the work.

- **References**: Lists the sources used to support the analysis.

2.4 Problem Statement & Context

2.4.1 Clear Problem Statement:

Patients discharged from hospitals often face challenges in maintaining proper follow-up and aftercare. Once they leave the hospital, many patients struggle to remember scheduled check-ups, follow the prescribed medication routines or monitor their recovery progress effectively. Without structured guidance, these gaps in care can result in missed appointments, incomplete treatments and even an increased risk of complications or relapse. For elderly patients, those with chronic conditions or individuals managing multiple prescriptions, the burden of remembering and coordinating post-discharge care is especially difficult.

Currently the lack of a reliable follow-up reminder system highlights a major weakness in patient care continuity. Hospitals may provide discharge instructions, but without ongoing reminders or check-ins, patients are left on their own to manage complex recovery processes. This contributes to higher rates of hospital readmission, increased healthcare costs and poorer health outcomes. By implementing a structured follow-up and reminder system—covering appointments, medication schedules and recovery milestones—hospitals can bridge this critical gap, ensuring that patients receive consistent support even after leaving the hospital.

2.4.1.1 Stakeholders and their expectations:

- **Patients**:Easy-to-use reminders for medications, follow-ups, and recovery tasks. Clear instructions in non-technical language. Notifications through preferred channels (SMS, app, call, WhatsApp, etc.). Privacy and security of personal health information. Personalized scheduling that adapts to their condition.
- **Family Members/Caregivers**: Ability to receive reminders if the patient forgets. Access to progress updates (e.g., whether the patient is adhering to medications). Alerts for missed appointments or potential health concerns.
- **Hospitals & Healthcare Providers**: Reduced readmissions and emergency visits. A dashboard to monitor patient adherence remotely. Better patient satisfaction scores and trust in hospital services.
- **Doctors / Nurses**: Confirmation that patients are following prescribed treatments. Notifications about non-compliance or missed milestones. Less time spent on follow-up phone calls/manual reminders. Data-driven insights into patient recovery trends.
- **IT/Admin Staff (System Operators)**: A reliable, secure, and easy-to-manage system. Minimal downtime and technical complexity. User support/helpdesk features for patients who struggle with tech.

2.4.1.2 BIS Relevance: An information system (IS) is any organized system that collects, stores, processes, and distributes information to support decision-making and control in an organization. This project qualifies as an information systems problem because it involves the integration of people, processes, and technology to manage post-discharge care effectively:

1. **People**: Patients, caregivers, and healthcare providers all rely on accurate, timely information to make decisions and take actions for recovery.
2. **Processes**: The system supports structured processes such as scheduling reminders, tracking medication adherence, and monitoring recovery milestones. Without an automated system, these processes would be manual, inconsistent, and error-prone.
3. **Technology**: The follow-up system uses technology to store patient data securely, process it to generate personalized reminders and distribute it through multiple channels (SMS, mobile apps, dashboards). Technology ensures efficiency, accuracy, and compliance with privacy regulations.

2.5 Theoretical Alignment

2.5.0.1 How the proposed system fits into BIS categories

- **TPS**: Its role is to store information such as the release date, scheduled appointments and reminder logs, automate tasks such as sending emails

through SMS, emails, recording attendance to follow up appointments and confirming each medication intake.

- **MIS**: Processes TPS data to generate reports for doctors and nurses and provides metrics like appointment attendance rates, medication compliance levels and readmission frequences.
- **DSS**: Helps doctors and managers make informed decisions through data analysis and prediction and identifies patients at higher risk of relapse or readmission based on follow-up data. Another function is allowing scenario simulations such as: "What if we increase reminders for patients over 70?"
- **ESS**: Provides top management with a strategic overview, summarizes key KPIs: reduction in readmission rates, cost savings, patient satisfaction improvements and supports investment and policy decisions.

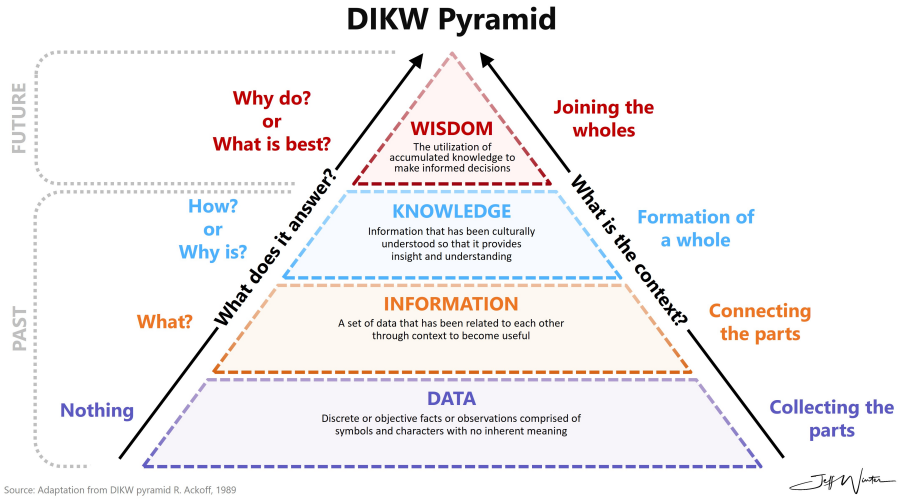


Figure 1: DIKW Pyramid

2.5.0.2 DIKW Framework: *Figure 1: DIKW Pyramid*

The DIKW hierarchy is a model that describes in detail how information is transformed from simple data into useful knowledge for strategic decision-making. The pyramid has four levels:

1. **Data**: They are raw facts without context. They only represent basic pieces of information, such as numbers, names, or dates.
2. **Information**: It is the result of organising and contextualising data. The information has meaning and allows us to understand the context of the data.
3. **Knowledge**: It arises from analysing information to identify relationships. It allows us to obtain a 'why' for the information.

4. **Wisdom**: It is the practice of knowledge for making strategic and effective decisions. It answers questions such as 'How to act' and 'Which is best'.

In conclusion, DIKW details how you can take raw data, transform it into useful information, generate knowledge from analysis, and finally use it to make decisions.

How the DIKW model is applied in our system:

DIKW Level	Meaning	Examples in our system	Actions/Decisions
Data	Raw facts without context	Appointment dates, patient names, prescribed medications, attendance records	Stored in the system database for later analysis
Information	Organized and contextualized data	Lists of patients with upcoming appointments, medication follow-up alerts, patients missing treatments	Helps medical staff identify patients needing additional attention
Knowledge	Analysed information to identify patterns and relationships	Older patients, those with chronic conditions, or on multiple medications are more likely to miss appointments or medications	Identification of high-risk patients and prioritization of personalized follow-up
Wisdom	Applying knowledge to make strategic decisions	Send personalized reminders via SMS or email, adjust follow-up frequency, plan preventive interventions	Improves treatment adherence, reduce readmissions, optimizes resources, and ensures continuity of post-discharge care

2.5.0.3 Type of organizational change: In this case, our project can be considered a process redesign organizational change, as it completely transforms the structure of the previous process. Traditionally, post-discharge care was a passive process where responsibility relied almost entirely on the patient. With our system, it becomes an active process in which medical staff can interact, supervise, and guide the patients through continuous reports on their condition, improvement, and state of the patient.

This redesign is justified because it not only increases patient safety and adherence to treatment but also reduces hospital readmissions and optimizes the use of the healthcare system, creating value both for patients and for the healthcare organization. .

2.6 Business Process Analysis

2.6.1 AS-IS Description: How the current process works

The process we described is working rigth now in the following way:

1. **Medical decision:** The doctor in charge of the patients must approve that the patient can continue recovering outside of the hospital.
2. **Summary discharge**: After this a discharge summary is filled and the steps the patient should follow for his recovery are settled.
3. **Instructions to the patient**: The patient gets written and verbal recommendations and instructions on how to properly end the recovery process.
4. **Follow-up appointment**: The doctor will make a future appointment for checking the patient's status. The patient has the responsibility of remembering the dates and make date changes manually
5. **Post-discharge phase outside hospital**: The patient must follow the instructions and remember them. There are not reminders and active monitoring of the hospital.

2.6.2 TO-BE Description: How the improved process will work

1. **Medical decision**: The doctor evaluates the patient as in the AS-IS process. The improvement is recorded automatically in the hospital system while the doctor fills in the diagnosis details.
2. **Summary discharge**: The system automatically generates a digital discharge summary with the diagnosis, treatments, medications, care instructions and follow-up task.
3. **Instructions to the patient**: Patients receive digital notifications (via app) with their instructions. Also, written and verbal instructions are still provided but now the system can choose to remind the patient more frequently.
4. **Follow-up appointment**: The system automatically schedules follow-up appointments and sends reminders to the patient.

2.6.3 Ishikawa Diagram

The current aftercare process has several problems. Patients often miss medication doses because they forget schedules or misunderstand instructions. This comes from relying only on verbal or paper notes, with no reminder system. Similarly, missed appointments happen when patients lose slips or are not reminded, since the hospital does not provide alerts.

There are also cases of incorrect medication usage, caused by unclear instructions and the lack of a digital tool to track dosages. These gaps often lead to hospital readmissions, as patients return with avoidable complications. The core reason is that there is no feedback loop or monitoring after discharge.

For doctors and nurses, this creates a high workload, as they must repeat instructions or handle preventable cases, largely due to manual processes. Patients

themselves feel anxious and unsupported, because there is no structured follow-up once they leave the hospital.

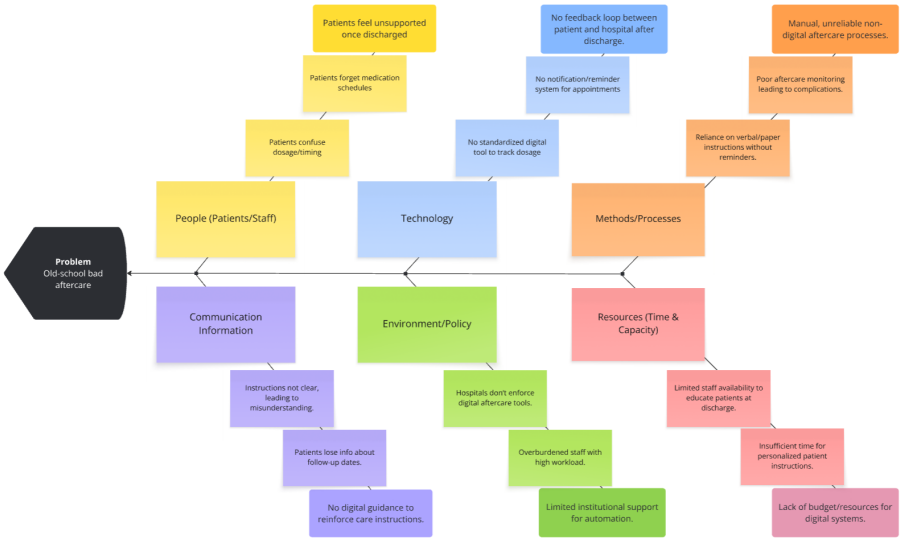


Figure 2: Ishikawa Diagram, fishbone

An improved aftercare process can be achieved by adopting digital tools for reminders, dosage tracking, and feedback, which reduce patient errors and missed appointments. Clear communication, structured follow-up, and automation ease staff workload, while proper resource and policy support ensure patients feel guided and supported, leading to fewer complications and readmissions.

2.6.4 BPMN Diagram

The process starts when a patient is considered ready for hospital discharge after an inpatient stay. The doctor evaluates the patient and decides:

- If the patient is not ready, the process ends because continued hospitalization is required.
- If the patient is ready but no follow-up is required, the process ends as well.
- If the patient is ready and follow-up is needed, the doctor approves the discharge and sends the discharge information to the system.

The system generates a digital discharge summary. Afterwards, the doctor schedules the next appointment and updates the system with the details. The system records the appointment and sends all discharge information to the patient.

The patient reviews the information:

- If the information is incorrect, the process ends with an error.

- If the information is correct, the process continues.

From this point, two actions occur in parallel:

1. The system sends daily notifications to remind the patient about their prescribed medication.
2. The system prepares appointment reminders depending on the scheduled date:
    - If the appointment is more than one week away, reminders are sent one week before and one day before the appointment.
    - If the appointment is less than one week away, only a reminder one day before is sent.

Finally, during the follow-up consultation, the doctor reviews the patient again. The process restarts from the beginning unless the initial conditions (not ready for discharge or no follow-up needed) are met, in which case the process ends.

BPNM diagram Camunda

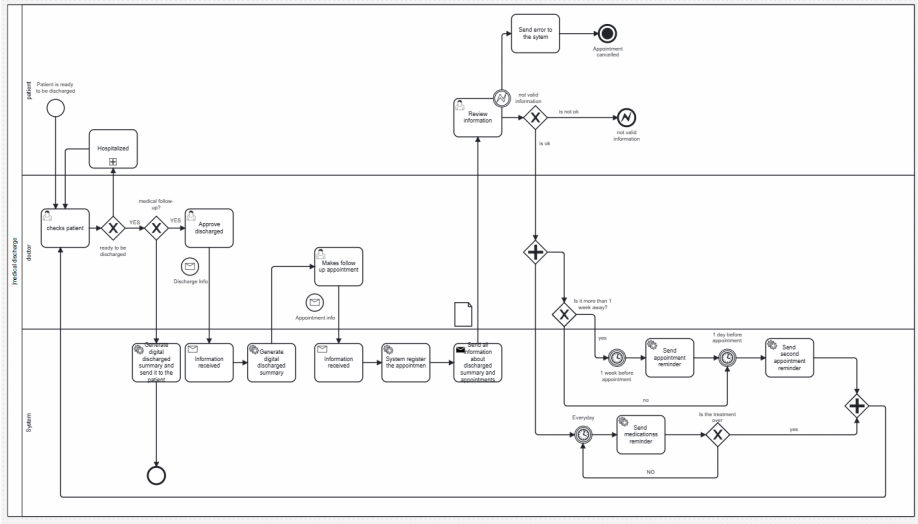


Figure 3: BPMN Diagram – Improved Aftercare Process

2.7 Database Connection

- **1. Discharge Decision by Doctor**

  Entity: Patient (patient ID, demographics, medical history).

  Entity: DischargeStatus (ready, not ready, no follow-up needed, follow-up needed).

  Database Action: The doctor's decision is recorded in the database under the patient's record.

- **2. Digital Discharge Summary**

  Entity: DischargeSummary (summary ID, patient ID, discharge date, diagnosis, prescribed medication, recovery instructions).

  Database Action: The system generates a new discharge summary entry and stores it in the database, linked to the patient.

- **3. Follow-up Appointment Scheduling**

  Entity: Appointments (appointment ID, patient ID, doctor ID, date/time, location, type of visit).

  Database Action: When the doctor enters the appointment details, a new appointment record is created in the database.

- **4. Patient Review of Information**

  Entity: PatientFeedback (feedback ID, patient ID, discharge summary ID, status: correct/incorrect).

  Database Action: The patient's confirmation (or error report) is logged in the database. If incorrect, the status flags the discharge summary as invalid and triggers review.

- **5. Medication Notifications**

  Entity: MedicationSchedule (schedule ID, patient ID, medication name, dosage, frequency, start/end dates).

  Entity: Notifications (notification ID, patient ID, type: medication/appointment, message content, send date/time, status: sent/delivered).

  Database Action: The system generates daily reminder entries in the Notifications table based on the MedicationSchedule.

- **6. Appointment Reminders**

  Database Action: The system checks the Appointments table:

  If appointment_date - today $>$ 7 days, insert two reminders (one at 7 days before, one at 1 day before).

  If appointment_date - today $<=$ 7 days, insert only one reminder (1 day before).

  These entries go into the Notifications table, linked to the appointment ID.

- **7. Follow-up Consultation**

  Database Action: During the follow-up, the doctor updates the patient's status in the DischargeStatus entity again. This restarts the process loop (new discharge summary, new appointment, etc.).

Every step of BPMN process corresponds to a database write, update, or query. That database becomes the backbone of the aftercare system, ensuring that data flows consistently between doctors, patients and the automated system.

An improved aftercare process can be achieved by adopting digital tools for reminders, dosage tracking, and feedback, which reduce patient errors and missed appointments. Clear communication, structured follow-up, and automation ease staff workload, while proper resource and policy support ensure patients feel guided and supported, leading to fewer complications and readmissions.

2.8 AI Collaboration Log

Chat log

https://chatgpt.com/share/68d41d21-cbb4-8000-a413-375121e5d508

- **Reflection:** AI was particularly helpful in structuring the problem, framing it from an information systems perspective and generating artifacts like a Lean waste analysis, an Ishikawa root cause map and even a Camunda-compliant BPMN model with example forms. These outputs saved time by providing ready-to-use templates, consistent terminology and links between concepts (discharge process → aftercare → IS solution).

  However, human intervention was needed to:

- **Decide scope and relevance:** AI generated broad options, but we had to choose which categories, tasks, and details fit the specific hospital context.
- **Validate practicality:** We need to filter BPMN steps, reminder, and form fields to match real workflows and patient needs.
- **Refine tone and purpose:** We adjusted the outputs depending on whether the audience was academic (problem statements), clinical (process issues), or technical (BPMN/JSON).

In short, AI accelerated the ideation and drafting phase, while we intervened to filter, adapt and contextualize the outputs for the real-world use case.

2.9 References

Covington, E. (2024, December 2). The DIKW Pyramid and the Process of Conducting an Advanced Review. Research Moment. https://researchmoment.unl.edu/the-dikw-pyramid-and-the-process-of-conducting-an-advanced-review/

3 Assignment 2 – From BPMN to Power Apps Prototype

3.1 Introduction

This project focuses on transforming a BPMN workflow into a functional Power Apps prototype for healthcare follow-up management system. The goal is to demonstrate how a process model can be systematically converted into interactive application features, connecting business logic with technical implementation.

The selected process represents the patient discharge and follow-up procedure in a healthcare environment. It includes key interactions between doctors and patients, such as recording discharge information, planning follow-ups, scheduling appointments, and sending reminders. By analyzing the BPMN diagram, the team identified the essential requirements and mapped them into system functionalities, user interfaces, and data structures.

The development follows an iterative approach, beginning with a Minimum Viable Product (MVP) and progressively adding more advanced features. Each iteration introduces new components — from basic forms and navigation to automated reminders and role-based access. This structured process ensures that the prototype remains aligned with the BPMN model while evolving into a practical and user-oriented application.

3.2 Mapping BPMN → Features & Data

Each BPMN task was translated into an application requirement, specifying the actions that the system must enable for users such as doctors and patients. Conditional process paths, such as whether follow-up is required or the timing of reminders, were also reflected in the requirements. Events such as patient discharge or sending reminders are represented as triggers that initiate specific system actions. Data handled within the process, including patient information, follow-up instructions, and appointment details, is captured as part of the system requirements to ensure accurate communication and follow-up. This approach ensures that each step in the BPMN has a corresponding and applicable requirement for the application.

Requirements identified from BPMN process:

- **R1:** The system allows the doctor to enter all patient discharge information.
- **R2:** If no follow-up is needed, it generates and sends a discharge report to thno e patient.
- **R3:** If follow-up is required, the doctor can record the treatment plan.
- **R4:** The system generates a summary of the follow-up report.

- **R5:** The doctor can schedule a follow-up appointment for the patient.
- **R6:** The system registers the appointment details.
- **R7:** The system sends the summary and appointment date to the patient.
- **R8:** The patient can review and confirm the received information.
- **R9:** If the information is incorrect, the system cancels the appointment.
- **R10:** If the appointment is more than a week away, reminders are sent one week and one day before.
- **R11:** If it's less than a week away, only a one-day reminder is sent.
- **R12:** The system sends daily medication remindersNotification

Requirement Mapping Table

BPMN Requirement	Feature (UI/Screen)	Data Needed (Entity/Field)	Notes
R1: Enter patient discharge info	Patient Discharge Form	Doctor(Id, Name), Patient(ID, Name), DischargeReport(Discharge_repId, Patient, Doctor, Needs_treatment, Diagnosis, Report_text)	Doctor inputs all discharge information
—	—	—	—
R2: Generate & send report if no follow-up	Automatic Discharge Report Generation	DischargeReport(Id, Patient, Doctor, Needs_treatment, Report_text), Notification(Id, Patient, Type, Content)	System automatically generates and sends report to patient
—	—	—	—
R3: Record treatment plan if follow-up required	Treatment Plan Registration Form	FollowUpReport(Id, Patient, Doctor, Treatment, Report_text)	Doctor records follow-up treatment plan
—	—	—	—

BPMN Requirement	Feature (UI/Screen)	Data Needed (Entity/Field)	Notes
R4: Generate follow-up summary	Follow-up Report Summary Generation	FollowUpReport(Id, Patient, Doctor, Report_text), Notification(Id, Patient, Type, Content)	System generates summary of follow-up report
—	—	—	—
R5: Schedule follow-up appointment	Appointment Creation Screen	Appointment(Id, Patient, Doctor, Date)	Doctor schedules follow-up appointment
—	—	—	—
R6: Register appointment details	Appointment Management Module	Appointment(Id, Patient, Doctor, Date)	System stores appointment details
—	—	—	—
R7: Send summary & appointment date to patient	Patient Notification System	Notification(Id, Patient, Type, Content), Appointment(Id, Date), FollowUpReport(Id, Report_text)	Sends summary and appointment info to patient
—	—	—	—
R8: Patient review & confirmation	Patient Review & Confirmation Interface	Patient(ID, Name), Notification(Id, Patient, Type, Content)	Patient can review and confirm received information
—	—	—	—
R9: Cancel appointment if info incorrect	Appointment Cancellation	Appointment(Id, Patient, Doctor, Date), Patient(ID, Name)	system cancels appointment
—	—	—	—
R10: Send reminders if >1 week away	Appointment Reminder	Appointment(Id, Date), Patient(ID, Name), Notification(Id, Patient, Type, Content)	Sends reminders 1 week and 1 day before appointment
—	—	—	—

BPMN Requirement	Feature (UI/Screen)	Data Needed (Entity/Field)	Notes
R11: Send reminder if <1 week away	Appointment Reminder	Appointment(Id, Date), Patient(ID, Name), Notification(Id, Patient, Type, Content)	Sends reminder 1 day before appointment
—	—	—	—
R12: Daily medication reminders	Medication Reminder	Patient(ID, Name), Notification(Id, Patient, Type, Content)	System sends daily medication notifications

3.3 Development Plan

3.3.1 General Scope

The project covers the basic elements of a healthcare discharge and followup management system. It includes the ability for healthcare providers to record discharge information, treatment plans, and appointment details, as well as to display notifications and reminders for patients. It focuses on manual data entry, basic record display, and simple navigation between screens because we dont have a database example like a hospital does. Automated report generation and data validation are not part of the current scope. These components are identified as possible future enhancements.

3.3.2 MVP scope

An MVP (minimum viable product) is the most basic and functional version of a product that is presented as a first version to validate an idea. Its purpose is to obtain real feedback from stakeholders and verify that it works before investing more resources (ProductPlan, n.d.).

In our case, the minimum requirements implemented in this first version are:

- **R1**– feature (patient discharge form)
- **R3**– feature (Treatment Plan registration Form)
- **R5**– feature (Appoinment creation screen)
- **R6**– feature (Appoiment management module)
- **R7**– feature (Patient notification system)

3.3.3 Features Excluded

- **R2**– feature (Automatic Discharge Report Generation)
- **R4**– feature (Follow-up Report Summary Generation)
- **R8**– feature (Patient Review & Confirmation Interface)
- **R9**– feature (Appointment Cancellation)
- **R10**– feature (Appointment Reminder)
- **R11**– feature (Appointment Reminder)
- **R12**– feature (Medication Reminder)

3.3.4 Iterations / Phases

It is defined as iterations, repetitive work cycles of fixed duration. In each iteration, the team plans, develops, tests, and reviews a defined set of product features (Scaled Agile, Inc., n.d.).

The development process is divided into three main iterations and follows an incremental, iterative approach. Each phase progressively adds new functionality, allowing the system to evolve from basic data entry to a more complete and automated follow-up tool.

3.3.4.1 Iteration 1 - Core MVP: Data entry and structure Setup In this iteration, the main goal is build the foundational structure of the applicaction, enabling basic data entry and navigation.

3.3.4.1.1 Focus requirements:

- **R1**– feature (patient discharge form)
- **R3**– feature (Treatment Plan registration Form)
- **R5**– feature (Appoinment creation screen)
- **R6**– feature (Appoiment management module)
- **R7**– feature (Patient notification system)

3.3.4.1.2 Main task:

1. Design and impleent the home scree with simple navigation
2. Create the patient discharge form (R1) for doctors to input dicharge information
3. Build the appointment creation screen (R5) and appointment management module (R6) to record an view appintment data.
4. Connect the app to a data source

The final deliverable is a functional MVP where users can enter dicharge, treatment and appintment information and send notificaicons to patients.

3.3.4.2 Iteration 2 - Enhanced navigation and data visualization

The main goal is improve usability and introduce basic record management and navigation features. - Focus requirements: R1-R7 (enhanced versions)

3.3.4.2.1 Main tasks:

1. Enable to record filtering and search options in the discharge reports, treatment plans and appintments.
2. Implement data validation rules
3. Refine styling and layout

The final deliverable will be a fully navigable app with connected screens, visual data display, and basic user interation improvements.

3.3.4.3 Iteration 3 - Future Enchancements (Automation and roles)

The main goal is extend system intelligence with automation, role-based access, and patient engagement tools.

3.3.4.3.1 Focus requirements:

- **R2**– feature (Automatic Discharge Report Generation)
- **R4**– feature (Follow-up Report Summary Generation)
- **R8**– feature (Patient Review & Confirmation Interface)
- **R9**– feature (Appointment Cancellation)
- **R10**– feature (Appointment Reminder)
- **R11**– feature (Appointment Reminder)
- **R12**– feature (Medication Reminder)

3.3.4.3.2 Main task:

1. Implement automatic discharge report generation (R2) and follow-up summary creation (R4).
2. Add patient-side interface for review and confirmation (R8) and appointment cancellation (R9).
3. Develop automated reminders for appointments (R10, R11) and daily medication notifications (R12).
4. Integrate role-based access control, distinguishing doctor and patient views.
5. Explore integration with external databases or APIs for real patient data management.

6. Conduct user testing and gather feedback for continuous improvement.

The final deliverable will be a complete, automated healthcare follow-up system ready for deployment and further scalability.

3.4 Project Backlog – Healthcare Follow-up System

This backlog lists all planned tasks organized by iteration.
Each iteration builds upon the previous one, following an incremental development approach.

3.4.1 Iteration 1 – Core MVP: Data Entry and Structure Setup

Goal: Build the foundational structure of the application with essential forms and data connections.

Task	Priority	Status	Notes
Build Home Screen with navigation	High	Done	Main entry point linking to all modules
—	—	—	—
Create Patient Discharge Form (R1)	High	[ ] To Do	Allows doctor to input discharge details
—	—	—	—
Create Treatment Plan Form (R3)	High	[ ] To Do	Used when follow-up is required
—	—	—	—
Develop Appointment Creation Screen (R5)	High	[ ] To Do	Enables doctor to schedule appointments
—	—	—	—
Build Appointment Management Module (R6)	Medium	[ ] To Do	Displays and manages appointment records
—	—	—	—
Implement Patient Notification System (R7)	Medium	[ ] To Do	Sends follow-up info to patients
—	—	—	—

Task	Priority	Status	Notes
Connect data source	High	Done	Configure local data table or SharePoint list
—	—	—	—
Test and verify data flow between forms	Medium	[ ] To Do	Ensure proper storage and retrieval

3.4.2 Iteration 2 – Enhanced Navigation and Data Visualization

Goal: Improve usability, introduce galleries, and add validation for user data.

Task	Priority	Status	Notes
Add Gallery View for submitted records	High	[ ] To Do	Display discharge and appointment data
—	—	—	—
Enable record filtering and search	Medium	[ ] To Do	Improve record management
—	—	—	—
Implement navigation flow between screens	High	[ ] To Do	Home → Forms → Records → Notifications
—	—	—	—
Add data validation rules	Medium	[ ] To Do	Validate required fields and formats
—	—	—	—
Apply consistent styling	Medium	[ ] To Do	Define colors, typography, spacing
—	—	—	—
Create reusable UI components	Medium	[ ] To Do	Navigation bar, form templates, etc.
—	—	—	—

Task	Priority	Status	Notes
Improve notification layout and readability	Low	[ ] To Do	Better presentation for patient messages
—	—	—	—
Conduct user testing	Medium	[ ] To Do	Verify usability and flow

3.4.3 Iteration 3 – Future Enhancements (Automation & Roles)

Goal: Extend automation, notifications, and user-role functionality.

Task	Priority	Status	Notes
Implement automatic discharge report generation (R2)	Medium	[] To Do	System-generated reports
—	—	—	—
Add follow-up summary generation (R4)	Medium	[ ] To Do	Auto-create follow-up summaries
—	—	—	—
Develop patient review & confirmation interface (R8–R9)	High	[ ] To Do	Patients can confirm or cancel info
—	—	—	—
Configure appointment reminder automation (R10–R11)	High	[ ] To Do	Sends reminders before appointments
—	—	—	—

Task	Priority	Status	Notes
Implement daily medication notifications (R12)	Medium	[ ] To Do	Sends recurring reminders
—	—	—	—
Add role-based access (doctor/patient)	Medium	[ ] To Do	Different permissions per user type
—	—	—	—
Integrate with external database or API	Low	[] To Do	For real-world scalability
—	—	—	—
Perform final usability tests and documentation	Medium	[] To Do	Prepare for deployment

3.4.4 Data Model

Entities & Attributes Table

Entity Name	Attribute	Data Type	Source	Notes
DischargeReport	Discharge_repId	Text	Generated	Primary key
DischargeReport	Patient	Text	Form	Required
DischargeReport	Doctor	Text	Form	Required
DischargeReport	Needs_treatment	Boolean	Form	Required
DischargeReport	Diagnosis	Text	Form	Required
DischargeReport	Report_text	Text	Form	Required
FollowUpReport	FollowUp_reportId	Text	Generated	Primary key
FollowUpReport	Patient	Text	Form	Required
FollowUpReport	Doctor	Text	Form	Required
FollowUpReport	Treatment	Text	Form	Required
FollowUpReport	Report_text	Text	Form	Required
Appointment	AppointmentID	Text	Generated	Primary key
Appointment	Patient	Text	Form	Required
Appointment	Doctor	Text	Form	Required

Entity Name	Attribute	Data Type	Source	Notes
Appointment	Date	Date	Form	Required
Notification	NotificationID	Text	Generated	Primary key
Notification	Patient	Text	Form	Required
Notification	Type	Text	Form	Required
Notification	Content	Text	Generated	Required
patients_medications	id	Text	Generated	Primary key
patients_medications	Name	Text	Form	Required
patients_medications	start_date	Date	Form	Required
patients_medications	patientInfos	Lookup	Form	Links to Patient
patients_medications	endDate	Date	Form	Optional

3.4.5 ER Diagram

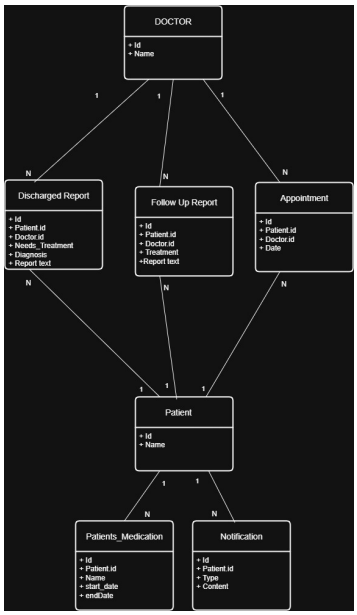


Figure 2: ER Diagram

3.5 App Prototype

3.5.1 Screens implemented

3.5.1.1 Log in screen Home screen (log in): navigation to log in as patient user or doctor user

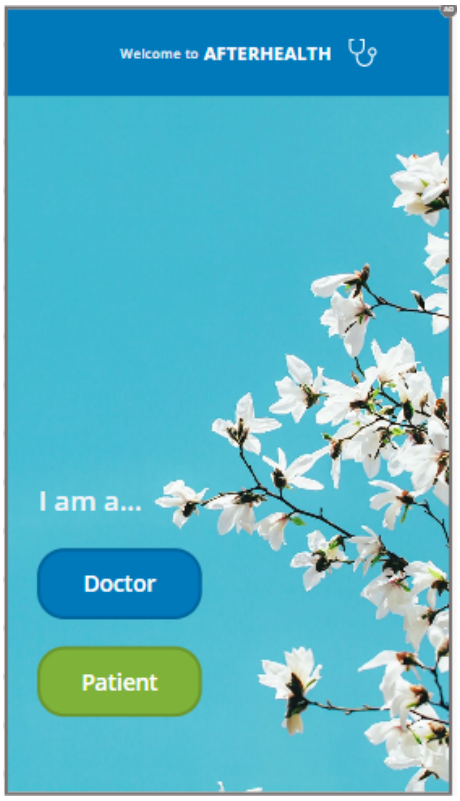
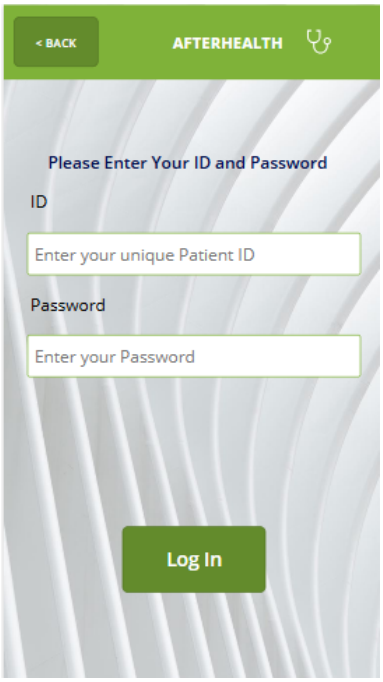
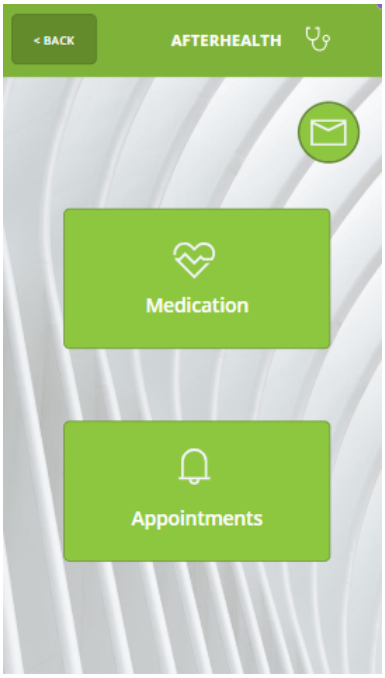
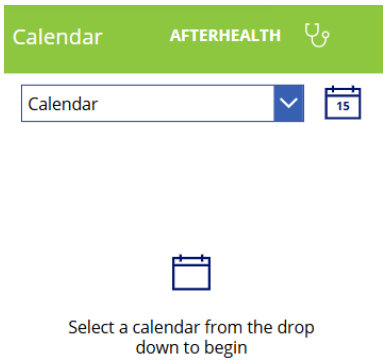


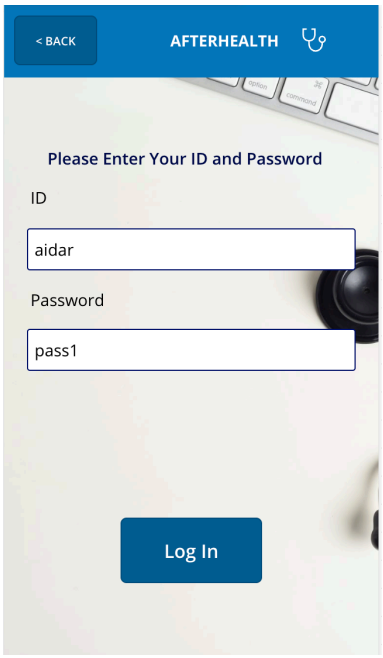
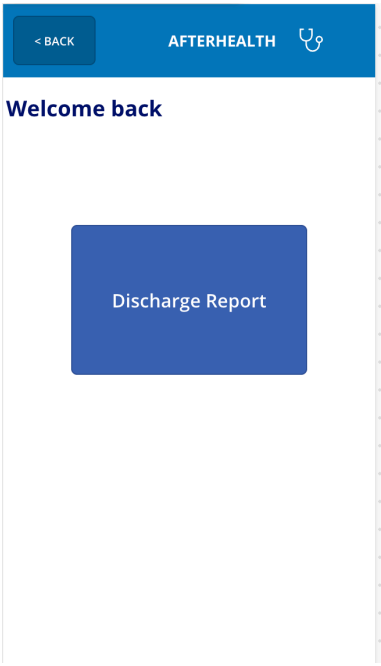
Figure 3: Log in screen

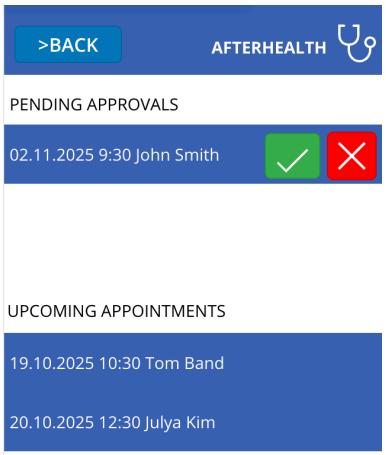
3.5.1.2 Patient's Screens Implemented

Patient Log-in	Patient Home Screen
	

Medication Screen	Appointments Screen
	

3.5.1.3 Healthcare Doctor's Screens Implemented

Doctor Login	Doctor Home
	

Discharge Report	Appointment Dashboard
	

3.5.2 Reusable Components

To ensure consistency and reduce duplication across screens, reusable components and containers were implemented:

3.5.2.1 Header/Navigation Component: This is the top-level component that provides both orientation and navigation across all screens. Its main purpose is to maintain consistency so users always know where they are and how to navigate back.

Features:

- **TitleText:** Displays the name of the app (AFTERHEALTH).
- **BACK Button:** Acts as a touchable logo or icon that returns users to the home dashboard.

Benefit: Users always see a familiar navigation and branding pattern, improving usability.

3.5.2.2 Separate Pages for Key Features: The application is structured into distinct pages, each dedicated to a specific user role (Doctor or Patient) and their associated functionalities. This separation ensures clarity and ease of use.

a) Login Screen Page:

Purpose:

Allows users to identify their role in the system — either as a Doctor or Patient — before accessing relevant functionalities.

Body Content:

Two main role-selection buttons:

- Doctor → Redirects to doctor-specific login or dashboard.
- Patient → Redirects to patient-specific login or dashboard.

b) Doctor Login Page:

Purpose:

Allows healthcare workers (doctors) to log in to their page by entering their ID and Password.

Body Content:

- Inputs for ID and password:
- Log in button

c) Patient Login Page:

Purpose:

Allows patients to log in to their page by entering their ID and Password.

Body Content: - Inputs for ID and password:

- Log in button

d) Doctor Home Page:

Purpose:

The Doctor Home Page allows healthcare workers (doctors) to access key patient management features. Its main purpose is to enable navigation to the Discharge Report screen, where doctors can view or update patient discharge details.

Body Content:

- A prominent, clearly labeled button that directs the user to the Discharge Report screen.

Button Text: "Discharge Report"

Function: On click, it redirects the doctor to the Discharge Report page.

e) Discharge Report:

Purpose: The Discharge Report page is part of the Doctor Home Page's patient management features. It allows healthcare workers (doctors) to access,

view, and eventually update patient discharge details. In future development, this page will be connected to the patient information database/table, enabling dynamic retrieval and update of patient records.

Future Enhancement:

- Integration with a patient information table to automatically load and display relevant patient details (e.g., patient name, ID, diagnosis, treatment summary, and discharge date).
- Support for updating and exporting discharge reports.
- Add filters and sorting options (by urgency, patient, or date).

f) Appointment Dashboard:

Purpose:

The Appointment Dashboard is a centralized interface designed for doctors to manage and approve patient appointments efficiently and to be aware of upcoming appointments. Instead of manually setting time slots, the system automatically schedules appointments based on the doctor's availability. The doctor's role is to review, approve, or reject these requests before the patient's visit.

Future Enhancement:

The Appointment Dashboard will be connected to a centralized database to enable:

- Real-time synchronization of appointment data.
- Automatic updates when patients book, modify, or cancel visits.
- Improved data consistency and scalability across the system.

g) Patient Home Page:

Purpose:

This screen serves as the main home or dashboard page for patients within the AfterHealth healthcare application. It provides patients with simple, direct access to key health management features — specifically their Medication details and Appointments.

Body Content:

Two large, easy-to-tap buttons are displayed prominently in the center of the screen:

- *Medication*

Function:
Takes the patient to their medication page, where they can view prescriptions, dosage schedules, and treatment information.

-*Appointments*

Function:
Opens the appointment management page where patients can view, schedule, or cancel upcoming visits.

h) Medications Page:

Purpose:

The Medications Page allows patients to easily access and manage their prescribed medications within the AfterHealth application. Its main purpose is to provide a clear overview of each medication's name, dosage, schedule, and instructions, ensuring patients can follow their treatment plans correctly and safely.

Body Content: List of prescribed medications

3.5.3 Overall Future Enhancements

The application is designed to be scalable and adaptable to future needs. Planned improvements include:

Expanded Data Availability

- Add more patient and doctor data to improve the comprehensiveness of the app.
- Include additional fields such as medical history, lab results, imaging reports, and care plans.

Improved Data Flow Across the App

- Ensure seamless synchronization between different pages (e.g., patient data updates automatically reflected in appointments, medications, and reminders).
- Enable real-time updates for both doctors and patients.

Advanced Navigation and Personalization

- Personalized dashboards for both patients and doctors based on role and usage history.
- Context-aware recommendations and alerts tailored to each user type.

Analytics and Insights

- Provide doctors with insights on patient adherence, missed appointments, and overall engagement.
- Enable patients to track their own health trends and medication adherence.

3.5.4 Data Connections

In this Power Apps healthcare application, we primarily used Dataverse as the main data source. Outlook was also integrated solely to provide a calendar function. We created separate data tables in Dataverse to manage different workflows, such as user login, appointments, and more. One challenge we encountered was that Dataverse code varied depending on each team member's browser language, but we successfully standardized it across all users. Currently, the data works without issues; however, if this app were to go live with real users, Dataverse would not be an ideal choice, as it is a free and limited platform.

3.6 Reflection & Lessons Learned

Learned how to use Power Apps thanks to the Microsoft Learn environment. We made the mapping before we started to do the app so it worked as a schematic for our assignment. There was the language issue with every group member using different base language for their explorers. We tried to manage without knowing how to fix that issue but after nearly we finished the project, we got a language error and everything turned black and gave us errors. So we created a new app and re-did everything we had done for the old one, it was a big time consumer. GitHub was used repeatedly in this assignment because we all need to have the updated same file to see and adjust the things we did. I (İbrahim AYDAR) personally did the login screens and looking up to data to see if the dataverse had that user. I also did two seperate register screens for Doctors and Patients but in the new version I couldn't put it thus we all had more important stuff to design in the app. Hopefully in the next step I will remake the register screens. Lastly for the teamwork experince, it was great because when we worked together on the app at the same time, we progressed so much than one people working. But I must say If our group members would tried to work on the app some days before, it would have been great for us. Cause those days some of us tried to complete the app and worked on it and after, with them trying to implement something, the language error occured once again. So my point is if they would have implemented before, our work on the other app wouldnt go to waste because we could have seen the error before and solved it before implementing so much stuff. Overall it was great, love my team. -İbrahim AYDAR

During this project, the team encountered challenges in maintaining full alignment throughout the development process. Some parts of the application were developed before all members had fully reviewed the documentation, including the development plan, requirements derived from the BPMN workflow, and the database design. As a result, certain features in Power Apps were implemented without fully reflecting the initial objectives and requirements, leading to inconsistencies and technical debt that needed to be addressed in subsequent iterations.

Despite these obstacles, the process was highly instructive. Mapping the BPMN diagram into concrete system requirements and translating them into functional screens in Power Apps strengthened our understanding of connecting business processes with application design. Additionally, designing the database and defining entities such as patients, doctors, appointments, and notifications highlighted the importance of structured data to ensure consistency across the app.

This experience emphasized the importance of clear communication, continuous review, and alignment with the project's goals. It also reinforced the value of thorough documentation and iterative development as a guide to minimize misunderstandings and maintain a coherent workflow throughout the project. - Sara Gutierrez

Throughout the course of this assignment, I was able to learn a lot about the Microsoft Learn environment and its tools, and I feel that I have gained significant technical knowledge. Before this assignment, I did not have much experience with low-code tools, but now I feel much more confident working with them. These tools definitely help save developers' time, and after spending a few hours learning how to use them, it becomes possible to build a functional app within several hours.

However, the process also came with its own challenges. As a team, we struggled to create a fully working application because the platform was new to all of us, and we had to solve several issues to make the system function properly. There is still room for improvement — such as implementing an efficient data flow, structuring the database more effectively, and creating proper relationships between different components.

As a team, I believe everyone was responsible and dedicated. While distributing tasks, each member did their best to complete their part successfully. Still, working in a team has its challenges — particularly when it comes to establishing clear communication, reaching common decisions, and expressing one's ideas effectively.

Overall, I believe this assignment provided a valuable learning experience. It taught me how to collaborate effectively in a team, listen to others' opinions, and work toward a shared goal. Even though the experience came with challenges, it was undoubtedly worthwhile. - Nigina Abdukhalimova

During this project, I learned a lot about using Microsoft Power Apps and how low-code environments can simplify the process of application development. At first, it was quite challenging to understand how the platform worked, especially when it came to connecting the app with the Dataverse and managing data between different entities aswell as some problems we had to deal with like formating, most of the app getting corrupted and even haveing to make the app again from scratch.

Personally, I contributed to the development of some of the app's screens and designing the overall plan. this assignment was a valuable learning experience

that strengthened both my technical and teamwork skills. I will add that one of the most difficult things was team comunication and organization, we definitly have to improve this for next assigment so this project goes better. - Alejandro Regidor Orellana

Me(Miguel Romero Castellanos) worked on points 2 and 3 of this document as well as creating the database tables and screens of the application involving the clients part and most of the doctors part such as appointment making and discharge forms, i think that this development has helped a lot in what establishing conections between screens and making some project management with the requirements and scope for example. - Miguel Romero Castellanos.

3.7 References

ProductPlan. (n.d.). Minimum viable product. ProductPlan Glossary. Retrieved from https://www.productplan.com/glossary/minimum-viable-product/

Scaled Agile, Inc. (n.d.). Iterations. Scaled Agile Framework. Retrieved November 26, 2025, from https://framework.scaledagile.com/iterations#what-is-an-iteration

4 Assignment 3 – DevOps Planning, Pipeline, and Automation

4.1 DevOps Planning (Azure Boards)

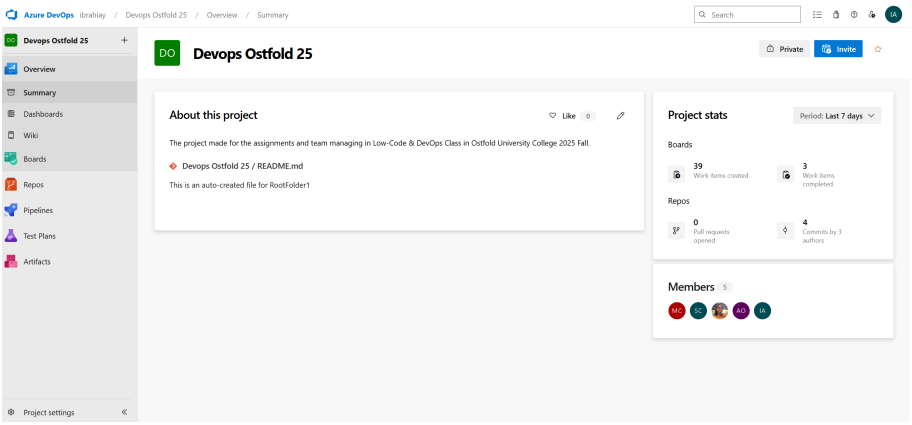
4.1.1 Project Setup

We created a solution in the powerapps named “AfterCare Solution”. Then logged in to the Azure Dev Ops environment and got the link to create the repository. After that went to Github Desktop and changed the account so it matches the Azure mail, while creating a main branch that we will use. After the creation, went to Azure to create a PAT key to publish the branch. Finally logged in to the PowerApps and connected the solution to the repositorys’ main branch.

4.1.2 Screenshot Overview

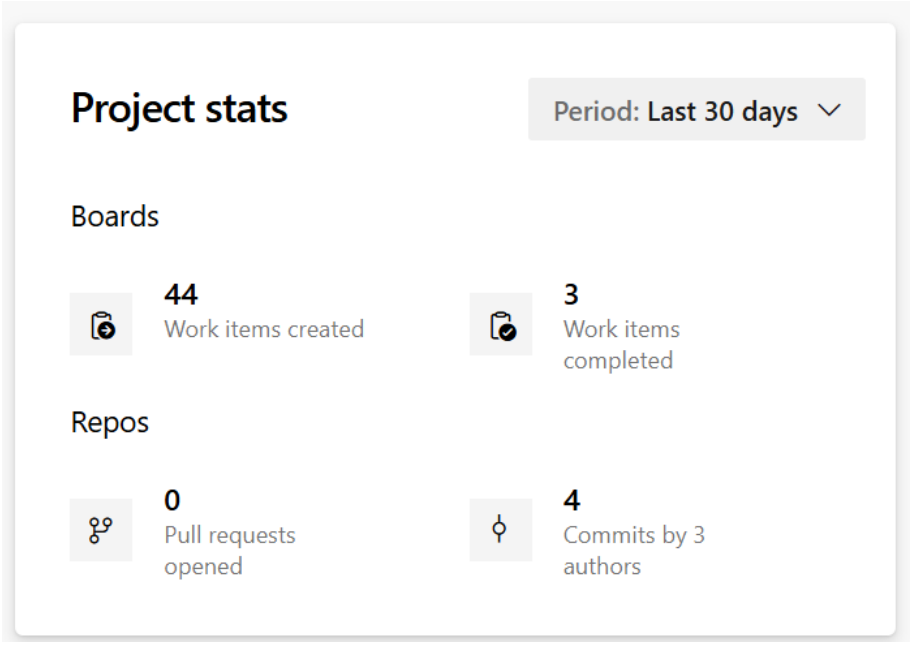
Last 7 Days

Figure: Overview of the Last 7 Days



Last 30 Days

Figure: Overview of the Last 30 Days



4.1.3 Work Items Definition

Epic	Description	Related Issues
AfterHealth App Is Fully Functional and Ready for Release	The entire application should operate smoothly end-to-end. All features, UI screens, and backend systems must be finalized, tested, and production-ready.	App features, backend integration, database schema, UI/UX screens, final bug fixes
—	—	—
Core Features Are Designed and Implemented	All essential features (e.g., profiles, tracking, appointments, notifications) should be fully designed, documented, and implemented with clear user flows.	Feature requirements, diagrams, feature dev tasks
—	—	—
Backend & Database Infrastructure Is Complete	Backend services, APIs, and the database schema must be fully designed, implemented, secured, and optimized for performance.	Database modeling, API endpoints, authentication, cloud deployment

Issue	**Description**	**Related Task(s)**
Patient wants to review and confirm the information	Create new Power Apps screen for confirming the information	retrieve patient data from database, synchronization, implement data validation
—	—	—
Doctor wants to make a summary generation of the follow-up report	**Button that gathers doctor's input and generates a summary report**	**add export/download option, design report data model, logic and functionality**
—	—	—
Doctor wants to make an automatic discharge report	Button which generates the discharge report	define discharge report data model, implementation of report generation service, data validation checks, download/export functionality

Issue	Description	Related Task(s)
—	—	—
Doctor wants to manage the appointments	**Screen where the doctor can perform CRUD functionalities**	**CRUD functionality, validation & logic, role-based access control, testing**
—	—	—
Patient & Doctor want to filter/search information	Filter settings	Backend logic, frontend/UI components, user experience improvements
—	—	—
Users want to have a better medication screen	**Minimal and functional medication screen redesign**	**requirements & design, logic & validation**
—	—	—
Manage the database	Check the relationships, data, and tables	verify forms and output data
—	—	—
The app must support different roles	**Create the login mechanism and role system**	**register screen for patient role & doctor role**
—	—	—
User must be notified	The patient should receive notifications for medication reminders	define notification use cases and triggers, choose notification channels, implement the service

Task	Assignee	Status	Notes
Testing	Ibrahim	To-Do	Testing functionalities from appointment management module
—	—	—	—

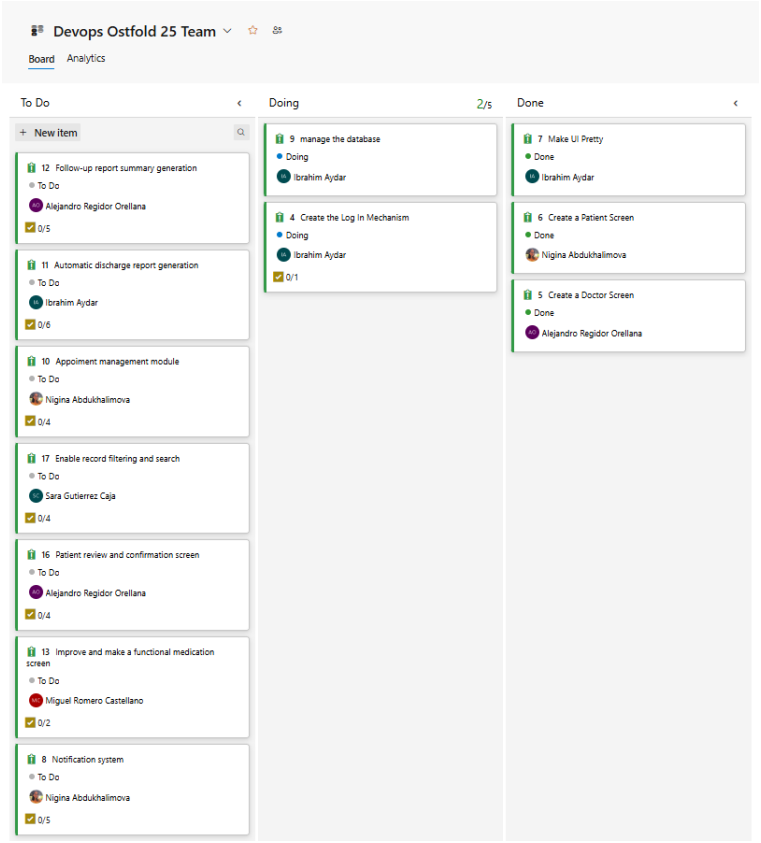
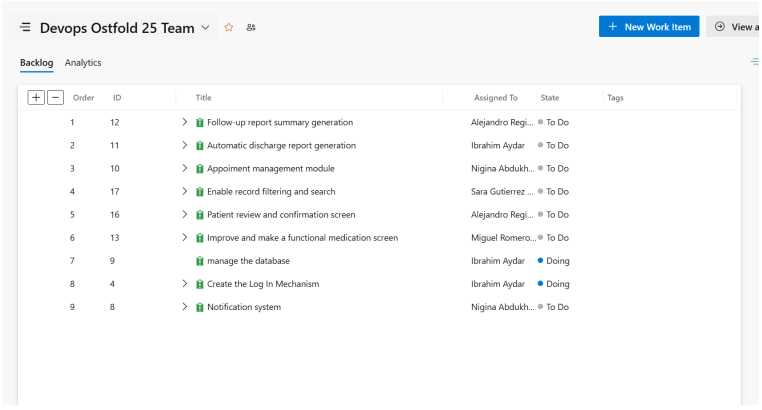
Task	Assignee	Status	Notes
Role-based access control	**Ibrahim**	**To-Do**	**Patients: manage their appointments; Doctors: view appointments assigned to them**
—	—	—	—
Validation & business logic	Nigina	To-Do	Validate overlapping appointments, prevent booking outside working hours, add status logic, handle time zones
—	—	—	—
Appointment CRUD functionality	**Nigina**	**To-Do**	**Create, Read, Update, Delete appointments (role-based views)**
—	—	—	—
Add export/download option	Miguel	To-Do	Download and export the report
—	—	—	—
Implement report preview functionality	**Miguel**	**To-Do**	**Preview the report**
—	—	—	—
Handle report storage and retrieval	Sara	To-Do	Store follow-up summary reports
—	—	—	—
Implement report generation logic	**Miguel**	**To-Do**	

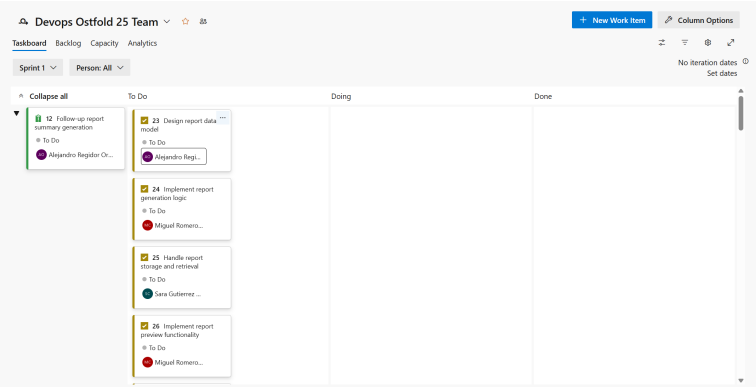
Task	Assignee	Status	Notes
—	—	—	—
Provide download/export functionality	Miguel	To-Do	Discharge report export
—	—	—	—
Design discharge report template	**Ibrahim**	**To-Do**	For discharge report generation
—	—	—	—
Implement data validation checks	Ibrahim	To-Do	For discharge report generation
—	—	—	—
User Experience (UX)	**Sara**	**To-Do**	**Loading states, clear filters, accessibility improvements**
—	—	—	—
Backend Logic	Sara	To-Do	Add filtering logic to DB queries
—	—	—	—
Testing	**Miguel**	**To-Do**	**Unit tests, integration tests, UI tests**
—	—	—	—
Frontend / UI Components	Ibrahim	To-Do	Search bar, filter panel, real-time search, dynamic results
—	—	—	—
Document the patient review workflow	**Alejandro**	**To-Do**	Patient review and confirmation screen
—	—	—	—
Create endpoint to retrieve patient data	Alejandro	To-Do	Patient review and confirmation screen

Task	Assignee	Status	Notes
—	—	—	—
Implement form validation	**Alejandro**	**To-Do**	Patient review and confirmation screen
—	—	—	—
Design patient review screen layout	Alejandro	To-Do	Patient review and confirmation screen
—	—	—	—
Validation & Business Logic	**Miguel**	**To-Do**	**Medication validation: dosage, dates, duplicates, conflicts**
—	—	—	—
Requirements & Design	Miguel	To-Do	Medication screen redesign, required fields, wireframes
—	—	—	—
Define discharge report data model	**Nigina**	**To-Do**	Automatic discharge report generation
—	—	—	—
Design report data model	Alejandro	To-Do	Follow-up summary report
—	—	—	—
Design notification center UI	**Ibrahim**	**To-Do**	Notification system
—	—	—	—
Implement notification service	Nigina	To-Do	Notification system
—	—	—	—
Select notification channels	**Nigina**	**To-Do**	Notification system
—	—	—	—

Task	Assignee	Status	Notes
Define notification use cases and triggers	Nigina	To-Do	Notification system
—	—	—	—
Register Screen for Both Sides	**Ibrahim**	**Doing**	Create the login mechanism
—	—	—	—
Implement message queue or scheduler	Nigina	To-Do	Notification system

4.1.4 Project Screens

Name	Screenshot
Boards	
Backlogs	

Name	Screenshot
Sprint	

4.2 Planning and Collaboration with Azure Boards:

4.2.1 Planning:

Planning was carried out in the same way as before, but it became much easier with the use of the Boards tab. This feature allowed us to quickly create tasks and subtasks, assign them to team members, and track their progress in a clear and organized way.

4.2.2 Collaboration:

This was the part that truly proved useful for our team. It allowed us to collaborate within a clear structure, always knowing what everyone was working on and how their progress was going. We no longer had to rely on constant communication for every small update, since Azure Boards automatically kept everything visible and up to date.

4.3 Power Apps Pipeline Setup

4.3.1 Environment Structure

In our project pipeline, three distinct environments were used to manage the Application Lifecycle Management (ALM) process — Development, Testing, and Production. Each environment serves a specific purpose within the pipeline to ensure controlled and systematic deployment of solutions.

Environment	Purpose	Characteristics	Role in Pipeline
Development	- Creation, configuration, and customization of the app.- Building and modifying Power Apps, flows, tables, etc.	- Contains **unmanaged** solutions (editable).- Used for design, testing, and debugging.- May include sample data or mock connections.	- Acts as the **source** for the pipeline.- Solution is exported from Development to Testing.
—	—	—	—
Testing	- Validate and verify functionality after deployment from Development.- Ensure app behaves as expected under realistic conditions.	- Uses **managed** solutions (not editable).- May include real or near-real data.- Has different environment variables and connection references.	- First **target** environment.- After successful tests, the solution can be promoted to Production.
—	—	—	—
Production	- Hosts the final version used by end users.- Only fully tested and approved solutions are deployed.	- Contains **managed** solutions.- Uses live data and secure connections.- Accessed by business users.- Often requires deployment approval.	- **Final stage** in the pipeline.- Deployment controlled by admins/approvers for governance.

4.3.2 Pipeline Configuration

Pipeline Stages

There are in total 3 stages in pipeline:

- Development (Source environment)
- Testing (Stage 1 target)
- Production (Stage 2 target)

4.3.3 Development Stage

The Development stage is the starting point of the whole lifecycle.
It works like a *workshop* where developers build and shape the solution.

What happens here:

- Developers create, configure, and customize the solution (such as apps, flows, and data tables).
- Everything is fully editable and flexible.
- Sample or mock data may be used to safely test ideas.
- Mistakes are expected here — it's a safe space to experiment.

Why it matters:

- It allows the team to design and test early versions before anyone else touches the solution.
- Nothing created here is final; it's all preparation for the next stage.

4.3.4 Testing Stage

Once the solution leaves Development, it is moved to the Testing stage.
This environment acts like a *quality control checkpoint*.

What happens here:

- The solution is deployed here from Development.
- It becomes a **managed** solution, meaning it cannot be directly edited.
- Testers or users validate that the solution behaves correctly under realistic conditions.
- The environment often uses data that looks like real business data to ensure accurate testing.

Why it matters:

- It ensures the solution is stable, reliable, and ready for real use.
- Problems must be detected and fixed here **before** reaching real users.
- Only solutions that pass testing move to the final stage.

4.3.5 Production Stage

The Production stage is the final destination.
This is the environment where the solution is fully live and used daily by real people.

What happens here:

- Only approved, fully tested solutions are deployed.
- Everything is deployed as **managed**, which prevents accidental edits.
- The system runs with live business data and is used by actual end users.
- Deployment is usually controlled or requires approval to ensure safety and compliance.

Why it matters:

- This is the environment that supports real business operations.
- Any mistakes here can impact the organization, so changes must be carefully controlled.
- Production represents the final, stable, and trusted version of the solution.

4.3.6 Pipeline

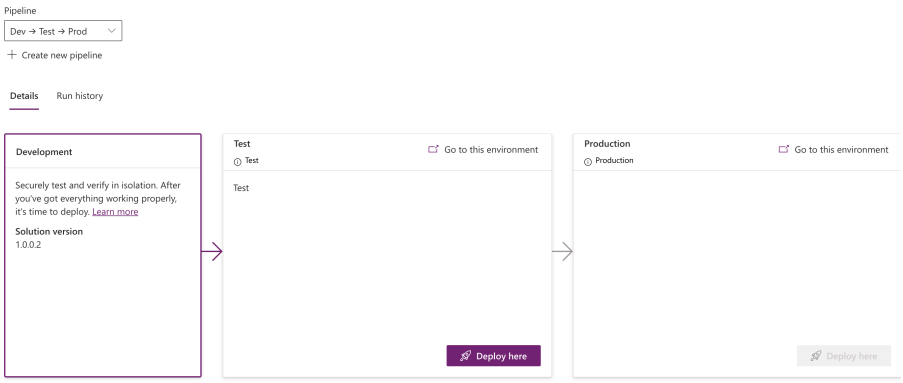


Figure 4: Pipeline

4.3.7 What does each stage do?

4.3.7.1 Development Stage: This is where the app or solution is created, customized, and tested by developers. All changes are made using unmanaged solutions.

4.3.7.2 Testing (UAT) Stage: The solution is deployed here as a managed solution for validation and user acceptance testing. It ensures the app works correctly before release.

4.3.7.3 Production Stage: This is the live environment where end users access the final, approved version of the app. No editing is allowed to maintain stability.

4.3.8 How is deployment triggered?

Deployment is manually triggered from the Development environment.A user selects the solution, clicks "Deploy", and chooses the next target environment (Testing or Production) Optionally, approvals can be configured so that deployment to Production requires authorization before proceeding.

4.3.9 How does this pipeline ensure version control?

The pipeline enforces version control by requiring each new deployment to use a versioned managed solution. Developers must increment the solution version number before each deployment, allowing clear tracking of changes and ensuring that older versions can be identified or rolled back if needed.

4.3.10 Lessons Learned

During the setup of the Power Apps pipeline, one key limitation identified was related to environment creation for the Testing and Production stages. Ideally, Power Platform allows different environment types such as Development, Sandbox, and Production. However, when configuring the pipeline, it was observed that new environments for Testing and Production needed to be created using the Development type rather than the typical Sandbox or Production types.

This limitation exists because the Power Platform Pipelines feature currently supports Dataverse-enabled Development-type environments for both the source and target stages of deployment. The pipeline host and target environments must meet certain prerequisites, including specific permissions and the presence of the Power Platform Pipelines solution, which are not automatically available in standard Sandbox or Production environments.

This restriction means that, although the environments are labeled as Testing and Production, they are technically classified as Development environments in the Power Platform Admin Center. This can lead to confusion when managing environments, as their official type does not reflect their actual purpose.

Additionally, during deployment to the Testing environment, an issue occurred. However after publishing the solution in Development Stage, the issue was resolved and everything worked smoothly.

This highlighted the importance of:

Publishing all the required solutions in the development Stage before deploying it in Testing Stage. Verifying connection references and environment variables to make sure they align with the target environment's configuration.

4.4 Automation with Power Automate

4.4.1 Flow Diagram

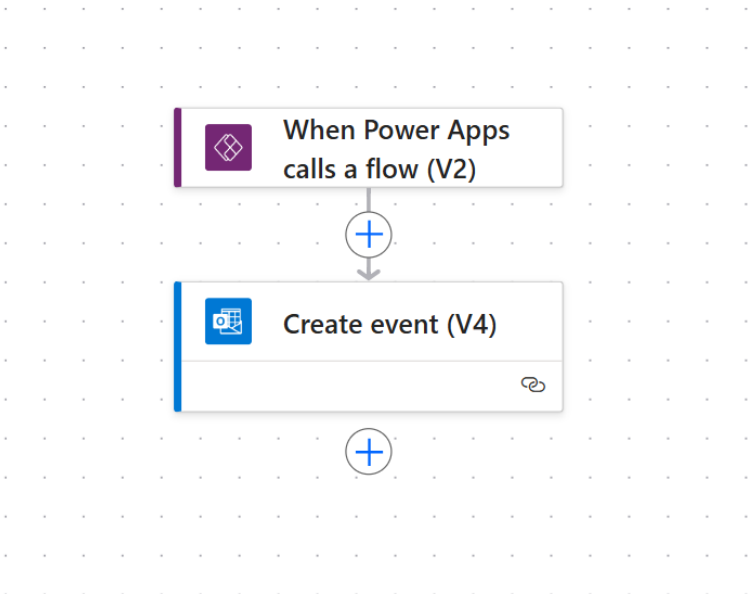


Figure 5: Diagram of the Flow

4.4.2 Flow Description

Our flow takes the data from the dataverse and creates daily reminders in calendar for you to take your medication. It has a starting and ending date.

Example:

We put 11th of June til 19th of November & choose the medicine Iburamin Cold, now I (Ibrahim AYDAR) have Iburamin Cold Medicine reminder every day in this interval.

Short Explanation: - It takes the medication chosen by the user and creates a reminder everyday in the interval that the doctor reported in the dataverse.

- This nearly solves the forgetting to take the medicine problem by reminding it everyday to the patient. Nearly because right now it is not fully automatical. It needs to take the order from the user, next versions will have automatic assignemnt for this.

- We created test time data in the dataverse and a name for the medicine, then used our mail to create reminders in outlook calendar.

Flow in BPMN: The project flow is integrated into the BPMN diagram, specifically in the 'Reminders' section.

4.5 Reflection

- **How did Azure Boards help organize work and improve efficiency?**

  Azure boards has helped us to organise, structure and distribute project tasks in a clear and organised manner. On the one hand, it allowed us to create epics, user stories and specific tasks for each team member. As a result, this facilitates planning, progress tracking and activity allocation.

- **What went well with pipeline deployment and automation?**

  It required significant effort from our team to properly configure the pipeline, as the process was complex and completely new to us. However, once it was successfully implemented, it operated smoothly and efficiently.

- **What were the main challenges and lessons learned?**

  The main challenge was adapting to the Azure DevOps environment and automating the flow, as it was completely new to us. We encountered several configuration issues throughout the process, which made it more difficult than expected.

  Another important lesson learned was the need to maintain a clear structure from the very beginning of the project—both in defining tasks within Azure Boards and in organizing the code—to facilitate collaboration and prevent conflicts during development.

- **How does this DevOps approach differ from your earlier development process?**

  The main challenge was adapting to the Azure DevOps environment and automating the flow, since we were completely unfamiliar with this platform. It was quite difficult at first because several configuration issues kept appearing throughout the process.

  Another lesson learned was the importance of maintaining a clear structure from the very beginning of the project — both in Azure Boards tasks and in the organization of the code — to facilitate collaboration and prevent conflicts during development.

---

5 Assignment 4 — Copilot Integration, Monitoring, Testing, and Production Deployment

---

5.1 Copilot Feature

5.1.1 Description and Purpose of CareMate-copilot feature

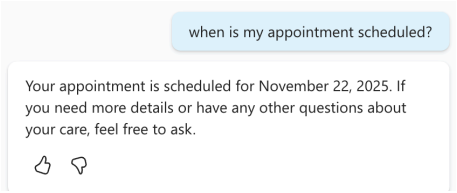
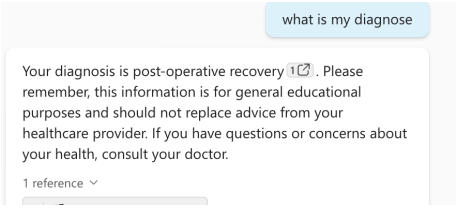
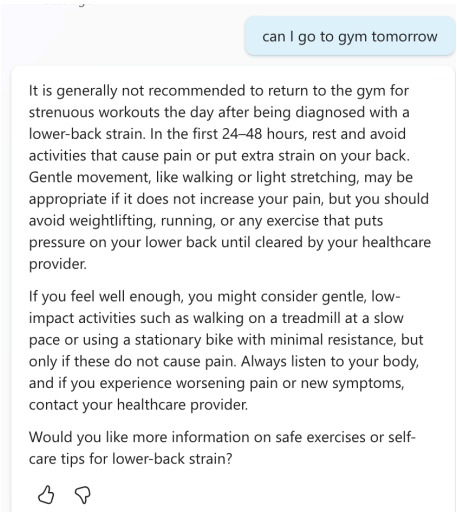
A chatbot is an AI-powered conversational agent integrated into the CareMate application to assist patients in managing their post-discharge care.

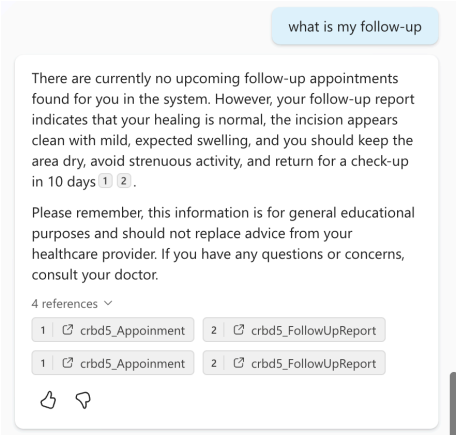
The CareMate Copilot's chatbot functionality enables users to quickly access clear, concise summaries of their post-discharge information stored across multiple Microsoft Dataverse tables. By connecting directly to key clinical data sources—such as DischargeReport, FollowUpReport, Patients, Appointments, and DoctorsInfo—the Copilot retrieves and presents the information each user needs in real time. In addition to data-driven responses, the Copilot leverages Large Language Models (LLMs) to provide general aftercare knowledge, patient education, and answers to common health-related questions. Using approved resources—such as Aftercare Guidelines and FAQs—the Copilot can deliver safe, easy-to-understand explanations that complement the user's clinical data.

This intelligent combination of structured data and LLM-powered knowledge significantly enhances in-app navigation. Users no longer need to manually browse through multiple screens; instead, they can directly request details such as recent reports, upcoming appointments, doctor information, or a consolidated overview of their recovery progress. The Copilot automates routine tasks such as filtering reports, summarizing clinical updates, and organizing follow-up instructions. By supporting immediate, context-aware queries, the Copilot increases efficiency, reduces cognitive load, and empowers users to make informed decisions based on both up-to-date medical data and reliable general health guidance. Ultimately, CareMate Copilot improves the patient experience by making post-discharge information more accessible, intelligent, and actionable.

Examples of interaction for different patients:

- When is my appointment scheduled?
- What is my diagnosis?
- Can I go to the gym?
- What are my follow-ups?

Category	Screenshot
Next Appointment	
Diagnosis	
Gym	

Category	Screenshot
Follow-up	

5.1.2 Knowledge Provided to the Copilot

- **Business context:** CareMate Copilot is designed to support patients after hospital discharge by helping them manage their recovery. It provides easy access to personal health information, tracks follow-up appointments, and offers guidance to ensure patients stay informed and engaged in their care.
- **Structured data:** The Copilot is integrated with Microsoft Dataverse tables, including DischargeReport, FollowUpReport, Appointments, and DoctorsInfo. These connections allow the Copilot to provide personalized summaries and respond to patient queries about their own reports and recovery status.
- **Conversational knowledge:** In addition to structured data, the Copilot uses Large Language Models (LLMs) combined with approved aftercare guidelines and FAQs to deliver safe, easy-to-understand explanations and guidance. This helps patients understand their recovery instructions and make informed decisions about self-care.
- **Security and permissions:** Patients have read-only access to their own records. The Copilot only displays information relevant to the logged-in patient, ensuring privacy and preventing access to other patients' data or sensitive clinical details not included in the summaries.

5.1.3 Data Connection

CareMate Copilot connects directly to Microsoft Dataverse to access and manage patient-specific information. Dataverse serves as the centralized data platform where all relevant patient records are stored, including tables such as

DischargeReport, FollowUpReport, Appointments, Patients, and DoctorsInfo. How the connection works:

- When a patient asks a question—such as "What is my next appointment?" or "Can I see my follow-up report?"—the Copilot queries the relevant Dataverse table using the patient's unique ID.
- The system fetches only the patient's own data, ensuring privacy and read-only access.
- Copilot then processes this data and generates a concise, personalized summary that is returned as a conversational response.

By using Dataverse, the Copilot can:

- Access structured patient data in real time.
- Provide accurate, personalized summaries of reports, appointments, and doctor information.
- Ensure that all patient interactions remain secure, with each user only seeing their own data.
- This integration with Dataverse allows CareMate Copilot to deliver intelligent, context-aware responses while maintaining strict data privacy and reliability.

5.1.4 User Interface design for chatbot

To provide a high level of user satisfaction, a simple and intuitive UI has been implemented on the patient's screen. Once the patient logs in, they can see the AI agent button at the bottom of the page. Clicking this button opens the chat interface, allowing the patient to start interacting with the AI immediately.

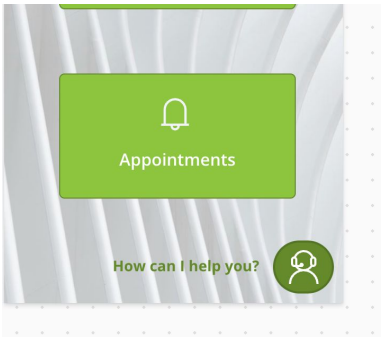
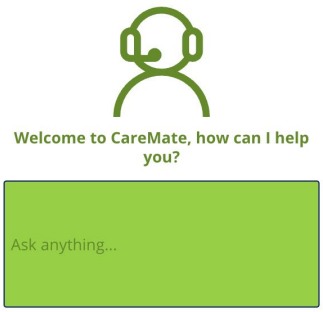
AI Agent	Chatbot Screen
	

Figure 2: UI for AI agent

- **Link to CareMate** https://copilotstudio.microsoft.com/environments/da76a5ad-5402-e460-9b93-677bc1061d56/bots/abad08cc-2abf-f011-bbd3-7c1e5272e83a/overview

---

5.2 Testing and Monitoring

5.2.1 Automated Tests

5.2.1.1 Doctor log-in failed

- **Name of test suites:** DOCTOR
- **Name of test case:** Case Doctor Log-in failed wrong user wrong password
- The test validates if the user introduces incorrectly their data, the system will not let it log in an the system will show again de log-in screen.

Step	Action / Control	Property / Value	Description
1	Button1	Select	Simulates a click on the first button (e.g., open login)
—	—	—	—

Step	Action / Control	Property / Value	Description
2	TextInput_ID	Text = “5555”	Enters an incorrect ID in the corresponding field
—	—	—	—
3	TextInput_Password	Text = “5555555”	Enters an incorrect password
—	—	—	—
4	Button2	Select	Simulates a click on the login button
—	—	—	—
5	Sleep	1000 ms	Pauses execution for 1 second to allow the app to update the screen
—	—	—	—
6	Assert	App.ActiveScreen = DoctorLogin	Verifies that the app remains on the login screen after invalid credentials

5.2.1.2 Doctor log-in test success

- **Name of test suites:** DOCTOR
- **Name of test case:** Case Doctor Log-in success
- The test validates if the user introduces correctly their data. If the data are correct, the system will continue its normal flow.

Step	Action / Control	Property / Value	Description
1	Button1	Select	Simulates a click on the first button (e.g., open login)
—	—	—	—
2	TextInput_ID	Text = “1”	Enters the correct ID in the corresponding field
—	—	—	—

Step	Action / Control	Property / Value	Description
3	TextInput_Password	Text = “kim1234”	Enters the correct password
—	—	—	—
4	Button2	Select	Simulates a click on the login button
—	—	—	—
5	Sleep	1000 ms	Pauses execution for 1 second to allow the app to navigate
—	—	—	—
6	Assert	App.ActiveScreen = DoctorScreen	Verifies that the app navigates to the DoctorScreen after valid credentials

5.2.1.3 Patient log-in failed

- **Name of test suites:** PATIENT
- **Name of test case:** Case Patient log-in failed wrong user wrong password
- The test validates if the user introduces incorrectly their data, the system will not let it log in an the system will show again de log-in patient screen.

Step	Action / Control	Property / Value	Description
1	Button1_1	Select	Simulates a click on the first button (e.g., open login)
—	—	—	—
2	TextInput_ID_1	Text = “1”	Enters the patient ID in the corresponding field
—	—	—	—
3	TextInput_Password_1	Text = “111112”	Enters an incorrect patient password
—	—	—	—
4	Button2_1	Select	Simulates a click on the login button

Step	Action / Control	Property / Value	Description
—	—	—	—
5	Sleep	1000 ms	Pauses execution for 1 second to allow the app to update
—	—	—	—
6	Assert	App.ActiveScreen = PatientLogin	Verifies that the app remains on the PatientLogin screen after invalid credentials

5.2.1.4 Patient log-in success

- **Name of test suites:** PATIENT
- **Name of test case:** Case Patient log-in success
- The test validates if the user introduces correctly their data. If the data are correct, the system will continue its normal flow for patient.

Step	Action / Control	Property / Value	Description
1	Button1_1	Select	Simulates a click on the first button (e.g., open login)
—	—	—	—
2	TextInput_ID_1	Text = “1”	Enters the correct patient ID in the corresponding field
—	—	—	—
3	TextInput_Password_1	Text = “hola”	Enters the correct patient password
—	—	—	—
4	Button2_1	Select	Simulates a click on the login button
—	—	—	—

Step	Action / Control	Property / Value	Description
5	Sleep	1000 ms	Pauses execution for 1 second to allow the app to navigate
—	—	—	—
6	Assert	App.ActiveScreen = PatientScreen	Verifies that the app navigates to the PatientScreen after valid credentials

5.2.1.5 Navigation

- **Name of test suites:** NAVIGATIONN
- **Name of test case:** Case all doctor screens
- The test validates of the screens from doctor's path

Step	Action / Control	Property / Value	Description
1	Button1	Select	Simulates a click on Button1 (e.g., open login)
—	—	—	—
2	TextInput_ID	Text = "5555"	Enter invalid ID in the field
—	—	—	—
3	TextInput_Password	Text = "1234"	Enter password in the field
—	—	—	—
4	Button2	Select	Click login button
—	—	—	—
5	TextInput_ID	Text = "555"	Update ID field
—	—	—	—
6	Button2	Select	Click login button again
—	—	—	—
7	Button5	Select	Simulates click on Button5 (example: navigate or select option)
—	—	—	—

Step	Action / Control	Property / Value	Description
8	Button3_5	Select	Simulates click on Button3_5
—	—	—	—
9	Button5_1	Select	Simulates click on Button5_1
—	—	—	—
10	Button3_6	Select	Simulates click on Button3_6
—	—	—	—
11	Button5_1	Select	Simulates click on Button5_1 again
—	—	—	—
12	Button3_6	Select	Simulates click on Button3_6 again
—	—	—	—
13	Button6	Select	Simulates click on Button6
—	—	—	—
14	Button7	Select	Simulates click on Button7
—	—	—	—
15	Button5	Select	Simulates click on Button5 again
—	—	—	—
16	Button3_5	Select	Simulates click on Button3_5 again
—	—	—	—
17	Button3_3	Select	Simulates click on Button3_3
—	—	—	—
...	...	...	Continue the pattern for remaining buttons

- **Name of test suites:** NAVIGATION
- **Name of test case:** Case all patient
- The test validates of the screens from patient's path

Step	Action / Control	Property / Value	Description
1	Button1_1	Select	Simulates a click on Button1_1 (e.g., open patient login)
—	—	—	—
2	TextInput_ID_1	Text = “1”	Enters the correct patient ID in the field
—	—	—	—
3	TextInput_Password_1	Text = “hola”	Enters the correct patient password
—	—	—	—
4	Button2_1	Select	Click login button
—	—	—	—
5	Icon3	Select	Simulates a click on Icon3 (e.g., navigation or action)
—	—	—	—
6	Button4	Select	Simulates a click on Button4
—	—	—	—
7	Button3_4	Select	Simulates a click on Button3_4
—	—	—	—
8	Button4_1	Select	Simulates a click on Button4_1
—	—	—	—
9	LblAppName1	Select	Simulates a click on label LblAppName1 (maybe navigation)
—	—	—	—
10	LblAppName1	Select	Click on label again (repeat)

5.2.1.6 Reports

- **Name of test suites:** REPORTS
- **Name of test case:** Case FollowUp Report
- The test creates a successful report

Step	Action / Control	Property / Value	Description
1	Button1	Select	Simulates a click on Button1 (e.g., open doctor login)
—	—	—	—
2	TextInput_ID	Text = “1”	Enters the doctor ID
—	—	—	—
3	TextInput_Password	Text = “kim1234”	Enters the doctor password
—	—	—	—
4	Button2	Select	Clicks the login button
—	—	—	—
5	Button5_1	Select	Clicks Button5_1 (e.g., navigate to prescription form)
—	—	—	—
6	DataCardValue8	Text = “Patient is good”	Fill in patient notes field
—	—	—	—
7	DataCardValue9	Text = “take 1 pill”	Fill in prescription instructions
—	—	—	—
8	DataCardValue10	Text = “1”	Fill in quantity/dose field
—	—	—	—
9	DataCardValue11	Text = “1”	Fill in another dose or repetition field
—	—	—	—
10	Button12	Select	Clicks submit/save button
—	—	—	—
11	Sleep	1000 ms	Waits 1 second to allow the app to process the submission

- **Name of test suites:** REPORTS
- **Name of test case:** Case DischargeReport
- The test creates a successful discharge report

Step	Action / Control	Property / Value	Description
1 Simulates a click on Button1 (e.g., open doctor login)	Button1	Select	
—	—	—	—
2	TextInput_ID	Text = “1”	Enters the doctor ID
—	—	—	—
3	TextInput_Password	Text = “kim1234”	Enters the doctor password
—	—	—	—
4	Button2	Select	Clicks the login button
—	—	—	—
5	Button5	Select	Clicks Button5 (e.g., navigate to medical form)
—	—	—	—
6	DataCardValue1	Text = “sick”	Fill in patient condition field
—	—	—	—
7	DataCardValue2	Text = “1”	Fill in quantity/dose field
—	—	—	—
8	DataCardValue4	SelectedItems = Table({‘Value’:{‘Id’:true,‘Value’:“Yes”}})	Selects an option in a dropdown/select field
—	—	—	—
9	DataCardValue4	Select	Clicks the dropdown field (confirm selection)
—	—	—	—
10	DataCardValue5	Text = “1”	Fill in another numeric field
—	—	—	—

Step	Action / Control	Property / Value	Description
11	DataCardValue6	Text = “he needs to take a pill all the days”	Fill in prescription instructions
—	—	—	—
12	Button11	Select	Clicks submit/save button

During the testing phase in Power Apps Testing, I encountered several difficulties. One of the main issues was the need to use **Sleep** before almost every validation to ensure that the app had enough time to update its state, otherwise the assertions would fail inconsistently. Additionally, I had to add specific **Assert** statements depending on the behaviour being tested, which required extra adjustments in each scenario. Another challenge was that we repeatedly needed to request permissions from the administrator, which slowed down the testing process and made it more difficult to carry out the tests smoothly.

5.2.2 Monitoring and Performance

We were only able to run Monitor from the Solution, not directly inside the app, but it still captured all events. There was a restriction for monitoring the app directly, even for the owner person.

Metrics Reviewed

- Network call duration
- Data load time (Filter, LookUp, Patch)
- Delegation warnings
- Screen/control rendering time

Findings & Improvements

- Delegation Warning:

  Filter(Appointments, Text(DoctorID)=Text(varDoctorID)) Fixed by removing unnecessary Text() conversion.

- Slow Data Loading:

  Large ClearCollect() calls reduced by filtering directly at the source.

- Multiple LookUps:

  Replaced repeated LookUp() with a single variable (varDoctorID).

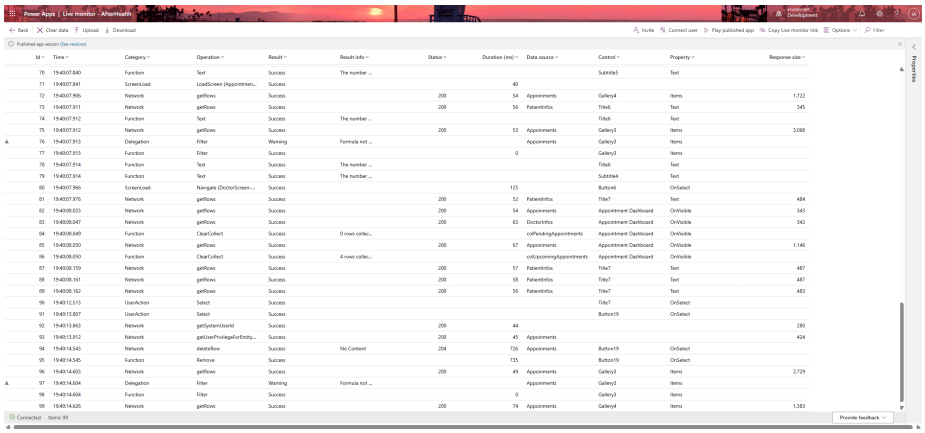


Figure 6: Monitoring

We reviewed duration(ms) and response size while live monitoring the app. There weren't any type of bottlenecks or errors so we continued.

Nontheless we improved the coding for dataverse to get rid of gibberish/unused parts.

5.3 Final Version Deployment to Production

5.3.1 Deployment Process

To ensure a controlled and reliable deployment, the application was moved through the Development -> Test -> Production environments using the Power Platform pipeline. This pipeline automates solution deployment and maintains consistency across all environments.

5.3.2 Deployment from Development to Test

- The initial version of the solution was built and configured in the Development environment.
- Once ready, the pipeline was triggered by selecting "Deploy to Test" from the Power Platform pipeline interface.

The pipeline automatically:

- Validated the solution components
- Packed the solution
- Imported it into the Test environment

In the Test environment, functional checks were carried out, including:

- Verifying data connections (Dataverse tables)

- Testing Copilot interactions
- Ensuring UI elements displayed correctly

5.3.3 Deployment from Test to Production

After confirming that the solution worked as expected in Test, the next step was to promote it to Production. The pipeline was triggered again using "Deploy to Production". The pipeline handled the import automatically while respecting solution dependencies and versioning.

5.3.4 Final Verification in Production

Once deployment was completed, the final version was tested directly in the Production environment to ensure everything functioned correctly. Verification steps included:

- Ensuring the Copilot responded to patient queries accurately
- Confirming the patient UI (screens, AI button, navigation) worked without errors
- Checking secure access to Dataverse tables (read-only data for each patient)
- Validating that reports, follow-up details, and appointments reflected real production data

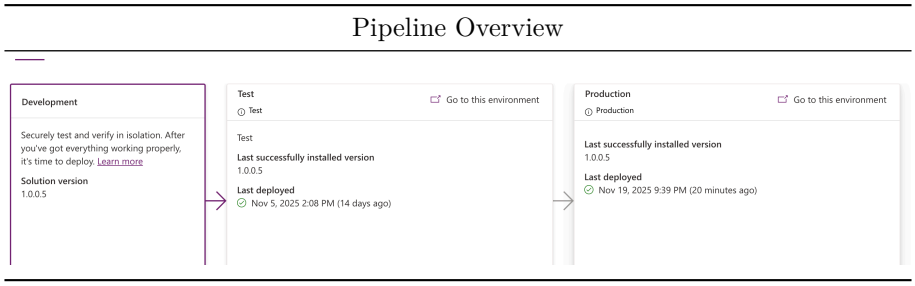


Figure 3: Successful deployment of pipeline stages

App in the Production Environment

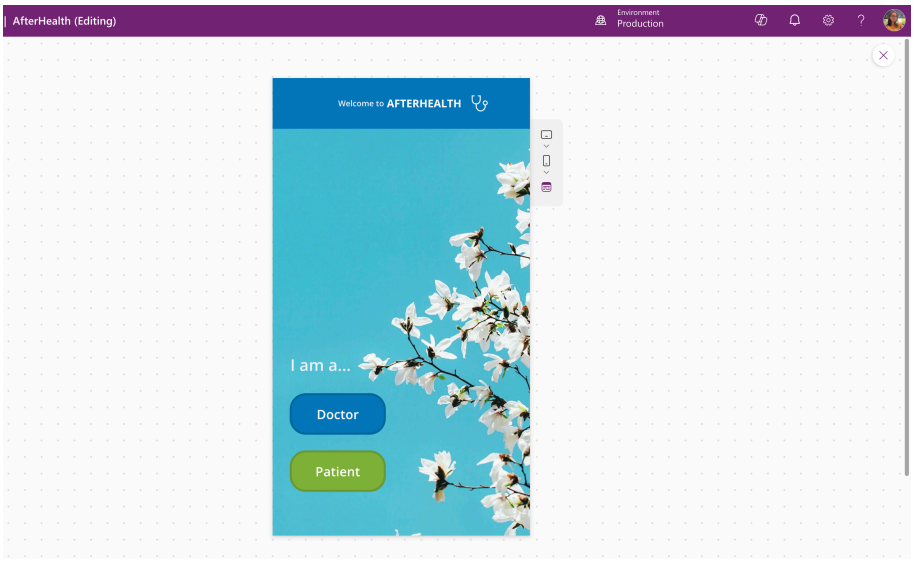


Figure 3: AfterCare App in Production Environment

5.3.5 Issues while deploying

During deployment using the Power Platform pipeline, the main issue encountered was related to missing solution dependencies. When promoting the solution from Development to Test, the pipeline flagged that several components—such as Dataverse tables and environment variables—were not included in the managed solution. To resolve this, all required components were added to the solution before re-running the pipeline.

This process highlighted the importance of verifying solution completeness before deployment and ensuring that any new components added during development are included in the exported solution.

5.4 Reflection and Lessons Learned

Integrating the Copilot into the application significantly transformed both the functionality and the overall user experience. The chatbot feature introduced a more intuitive way for patients to access their health information, reducing the need for navigating multiple screens and enabling more immediate, conversational interactions. This shift not only improved efficiency but also made the app feel more accessible and supportive during the post-discharge process.

Throughout the development of automated tests, I learned the importance of ensuring that the system reacts consistently to user input. The testing phase

highlighted practical challenges—such as the need to include waiting periods before validations and the requirement to adapt Assert statements depending on each scenario—which helped deepen my understanding of how the app behaves under automation. These insights allowed me to refine the logic of certain screens and confirm that the critical paths of the application worked reliably.

The deployment pipeline also played an essential role in the project. Automating the promotion of solutions across environments provided a smoother and more structured DevOps workflow, reducing manual errors and ensuring that updates were delivered in a controlled way. By combining AI integration, automated testing, and pipeline-based deployment, this project demonstrated how modern development practices can enhance both the reliability and the scalability of Power Platform applications.

Although the application has progressed significantly, there are still some features and refinements that could not be implemented at this stage. These missing elements will be addressed and expanded in a future iteration to further improve functionality, performance, and the overall user experience.

6 Final Reflection

6.1 Executive Summary

- What problem does your app solve?
  The main goal of our app is to give patients and doctors a platform to better manage and monitor discharge instructions after a hospital stay. It aims to improve communication between patients and healthcare providers, ensuring that patients understand and follow their discharge instructions effectively.
- Who are your stakeholders or target users?
  Our primary stakeholders are patients who have been discharged from hospitals and healthcare providers, including doctors and nurses, who need to communicate discharge instructions effectively.
- What business outcomes or improvements does your app enable?
  The app aims to reduce readmission rates by ensuring patients adhere to their discharge instructions, improve patient satisfaction through better communication, and streamline the workflow for healthcare providers by providing a centralized platform for managing discharge information.

6.2 Future Work

If we had more time we would add:

1. **Enhanced AI Integration:** Implement more advanced AI features for personalized patient care recommendations based on individual health data.

2. **Telehealth Integration:** Add telehealth capabilities to allow patients to have virtual follow-up consultations with their healthcare providers.
3. **Multilingual Support:** Expand language options to cater to a broader range of patients, ensuring accessibility for non-English speakers.
4. **Wearable Device Integration:** Integrate with wearable health devices to monitor patient vitals and provide real-time feedback to both patients and healthcare providers.
5. **Comprehensive Analytics Dashboard:** Develop a more robust analytics dashboard for healthcare providers to track patient outcomes and identify trends in discharge compliance.

6.3 Detailed Teamwork Reflection

Details about each team member's contributions, collaboration, and reflections:

6.3.1 Member 1 – Alejandro Regidor Orellana

Main responsibilities:
My main responsibilities included a bit of everything including:

- Managing the Git repository and making sure the report was up to and correct.
- Designing and implementing several screens in the Power App.
- Doing a large part of the testing and debugging of the app.
- Ensuring proper organization and task distribution among team members.
- Implementing some screens and features in the Power App.

Development contributions:
I implemented the following features and screens in the Power App:

- Top-bar navigation for easy access to different sections of the app.
- Patient Dashboard screen
- Doctor Dashboard screen
- Making sure everything followed a consistent design and user experience.
- Tests
- Some data model design and implementation.

Additionally, designing the app i did:

- Bussiness process analysis

- Database schema design
- BPMN aswell as doing implementing it as feautures and data.

Also, I contributed to the documentation and reporting of our project.

Collaboration:
Most of the support came in the form of helping with Git and DevOps related issues, steping in to help with their tasks when they were stuck or needed assistance, and ensuring that everyone was on the same page regarding project goals and deadlines. Aswell as coordinating meetings and discussions to facilitate effective communication within the team.

Reflection:
I learned a lot about Power Apps and the Microsoft Power Platform in general. I also gained experience in project management and teamwork, which are crucial skills in any development project. One of the main challenges we faced was coordinating our efforts and ensuring that everyone was contributing effectively. We overcame this by establishing clear communication channels and regular check-ins to discuss progress and address any issues that arose. Additionally, technical challenges related to app development were addressed through collaborative problem-solving and leveraging online resources and documentation. Most of the techincal challenges were solved by doing microsoft certifications.

6.3.2 Member 2 – Sara Gutierrez Caja

Main responsibilities:
Throughout this project, my responsibilities have involved contributing to multiple areas of the workflow. Some of the key aspects of my role include:

- Ensuring, together with my team, the proper use of GitHub version control.
- Managing final merges across the different branches.
- Designing and implementing key components within the Power App application.
- Performing testing and debugging to maintain functionality and quality.
- Making sure that the documentation remains consistent with the actual state of the application.
- Overseeing the workflow by helping divide and organize tasks effectively.

Development contributions:
During the implementation, I prepared the documentation of each part of the project, explaining every element of the work in detail.

- Theoretical Alignment Point
- Business Process Analysis

- BPMN Diagram Design and Modeling
- Database Design
- Mapping BPMN to Features & Data
- Development Plan & Project Backlog
- ER Data Model
- Design of several app prototype screens (patient, medication, appointments) and their fuctionalities
- DevOps Planning
- Planning and collaboration using Azure
- Testing & monitoring

Collaboration:
The collaboration overall has been good. At the beginning, some team members did not contribute as expected, so it was a bit difficult to divide the work and carry it out with good quality, especially since the workload for each team member was not balanced. Despite this, during development we all became more familiar with the environment, which led to clearer and more consistent tasks that we could distribute more effectively. It's worth noting that many parts were modified and worked on simultaneously by several people, since often someone would get stuck and another person would help solve the issue—resulting in all of us, at least to some extent, working on everything.

Reflection:
I have mainly learned about Microsoft Power Apps, Power Automate, and Copilot. Thanks to this course, I learned how to solve problems that came up during development, which allowed me to build an initial foundation that will be useful in the future. Regarding teamwork, I learned to be more patient and empathetic with certain teammates, and I also learned to communicate when disagreements arose, as well as to provide ideas and be more flexible with other people's perspectives.

The first major challenge we faced was organization. At the beginning, we organized ourselves using tasks and deadlines, but some people did not complete the assigned tasks or were absent without giving any response. This caused the rest of the team to be unable to work with the desired efficiency and effectiveness.

On the other hand, the application gave us many unexpected issues, such as each of us having a different default browser language, difficulties during testing, and even problems when implementing certain features. These problems were mostly solved through one key component: communication. We established certain agreements that led to everyone working and contributing more consciously. Meanwhile, implementation issues were resolved by reading documentation and slightly adjusting the functionalities we had planned in order to better adapt them to PowerApps.

6.3.3 Member 3 – Miguel Romero Castellano

Main responsibilities:
Throughout this course, my responsibilities have contributed to multiple areas of the project. Some of the contributions include:

- Designing and implementing key components within the Power App application such as most of the screens and dataflows to ensure all the functionalities work as expected.
- Some work in the development of the copilot bot.
- Database creation and management.

Development contributions:
Theoretical Alignment Point

- Business Process Analysis
- BPMN Diagram Design and Modeling
- Database Design
- Mapping BPMN to Features & Data
- Development Plan & Project Backlog
- ER Data Model
- Design of several app final screens (all the appointments related screens and notifications screens) and their fuctionalities
- Planning and collaboration using Azure

Collaboration:
Overall, the collaboration has been positive. At the beginning, some team members didn't contribute as much as expected, which made it a bit challenging to divide the work and maintain good quality, especially since the workload wasn't evenly balanced. Even so, as we progressed, we all became more comfortable with the environment, which allowed us to define clearer and more consistent tasks that we could distribute more effectively. It's also worth mentioning that several parts were modified and worked on simultaneously by different people, since someone would often get stuck and another person would step in to help—resulting in all of us, to some extent, working on almost everything.

Reflection:
In this project, I familiarized with all the microsoft development enviroment(Power apps,Copilot Studio, Dataverse…) while i was able to practice some skills and learn new ones such as BPMN diagrams management and Project workflow management. When tanlking about difficulties I found, I have to say that it was quite difficult in the beggining as I had never used any of this microsoft enviroment apps so i had to learn on the way while

working, but in the end I have been able to learn how to use them efficiently and delievr a quality work. —

6.3.4 Member 4 – Nigina

Main responsibilities:

- Implementing some screens and features in the Power App.
- Built and managed the Dev → Test → Prod pipeline and successfully deployed the model across all pipeline stages.
- Preparing a detailed report describing the functionalities and workflow of the app.
- Designing the Chatbox and Chat Page UI, ensuring a user-friendly and intuitive interaction experience.
- Integrating a Custom Copilot using Microsoft Copilot Studio to enhance automation and user engagement.
- Collaborating with the team by assisting with testing, debugging, and improving overall application performance

Development contributions:
I have implemented the following:

- Developed the home page of Doctor Screen, including UI design, layout structure, navigation flow
- Designed and implemented the Chatbox interface and the full Chat Page UI
- Worked on several app functionalities such as data handling, user interactions and linking UI elements with backend logic.
- Built and managed the Dev → Test → Prod deployment pipeline successfully, deployed the model/app through all pipeline stages and handled configuration and version updates.
- Integrated a Custom Copilot using Microsoft Copilot Studio

Collaboration:
Working with my group was a great experience. As exchange students from different countries, we struggled with communication at first but quickly learned to collaborate well. I encouraged face-to-face meetings. Whether my group mates were traveling or I was away, I made sure to stay in touch with the group and understand their availability or unavailability. I respected everyone's opinions, tried to make sure that everyone was being heard and that everyone's opinion is equally important.

Reflection:
DevOps and Low-Code Development have definitely been one of my favorite

subjects this semester. Before this course, I had no practical experience, except for a Software Development course at my home university, which mostly focused on theory. Therefore, I had little knowledge of how software development approaches could be applied in real-life applications. This course, however, was not only useful for learning and applying DevOps to an application but also for actually building a working Low-Code application.

At the beginning, it took quite some time to get familiar with all the Microsoft Low-Code tools. Even in the early stages, we had to build the UI for the app three times: first using a template, then facing issues with language settings because everyone's PC was in a different language and finally, on the third attempt, we learned from our mistakes and successfully completed it. Later, it was very rewarding to work with Power Apps, Power Automate, and all the Microsoft tools, as they significantly reduced development time.

However, not everything ran smoothly. While deploying a version of the app through the pipeline stages, I initially could not deploy it and encountered an error. Eventually, I solved the issue by making sure the latest version of the app was published in the Development Environment, which allowed successful deployment in the Testing Environment. Even during the final assignment, while integrating the Chatbox, I did not know that it is important to train the chatbot with the proper data tables, so it kept generating generic LLM answers. For example, when I asked, "When is my next appointment?" it responded, "I do not have access to that data; ask your medical advisor." While it might sound funny, it highlighted a real issue. I resolved it by training the chatbot with more relevant data and adjusting the question prompts so it could provide clear and accurate answers.

Throughout the project, there were many errors that we, as a group, had to solve. Fortunately, we were able to collaborate effectively, handle issues and learn from each challenge. Personally, I learned how to collaborate with people I had never worked with before, gained confidence using Low-Code tools and understood how software development processes can be applied in real-life projects. Most importantly, I realized that mistakes are an unavoidable part of the software development cycle, but with the right team, skills, determination and a bit of patience every problem can be solved.

6.3.5 Member 5 – İbrahim AYDAR

Main responsibilities:

- Making sure dataverses are correctly implemented.
- General UI and color scheme.
- Implementing automation using Power Automate.
- General Coding

Development contributions:
I mostly worked on code side and UI of the app.

- Color and background choices are made as non brutalist so everyone should feel welcome.
- The small automations to send a mail or adding reminders to the calendar is made by me.
- Login Screen, DoctorRegister, PatientRegister, DoctorLogin, PatientLogin and some stuff on the AppointmentDashboard screens are also made by me.
- Monitored how the app performs and did improvements to the codes.
- Monitoring

Collaboration:
Our team worked well together, and I'm genuinely proud of what we accomplished. We quickly built strong communication and a supportive working atmosphere. Everyone contributed openly, respected each other's ideas, and stayed committed to our shared goals. We made decisions together, kept each other updated, and helped one another whenever needed. Overall, our teamwork was smooth, positive, and genuinely enjoyable. I couldn't have asked for a better group.

Reflection:
Working on this project taught me a lot about both technical development and collaborative problem-solving. I learned how to structure Dataverse tables effectively and connect them seamlessly to the app, which strengthened my understanding of data management in Power Apps. Designing the UI and selecting an accessible color scheme helped me think more carefully about user experience and inclusivity. I also gained hands-on experience with Power Automate by creating flows for emails and calendar reminders, which improved my automation and workflow-building skills. Throughout the project, I improved my coding practices, learned how to identify performance issues, and refined the app through continuous testing. Overall, this project helped me grow as a developer and better understand how different components of a low-code system come together to form a functional application. —

6.4 Sign-off

All team members confirm that: - The work submitted is original and collaborative.
- Each member's described contributions are accurate.

Signatures / Names:
- Alejandro Regidor Orellana - Sara Gutierrez Caja - Miguel Romero Castellano - Nigina - Ibrahim - 02/12/2025